BestilVasketid.dk

Svendeprøve til Datatekniker med Speciale i Programmering

af FCH GROUP 2020

Frederik Christensen, Casper Hilligsøe & Henrik Poulsen

Technical Education Copenhagen, Ballerup

Procesrapport

Vejleder: Jan Stern Johansen & John Ellehammer

Afleveringsfrist for kravspecifikationen 6/11 -2020

Afleveringsfrist for projekt 20/12 -2020

Indholdsfortegnelse

1 Hvem er vi? 4
1.1 Frederik Christensen 4
1.2 Casper Hilligsøe 4
1.3 Henrik Vincents Poulsen 4
2 Indledning 5
3 Problemformulering 5
4 Projektbeskrivelse 6
5 Projektstyring 7
5.1 Discord 7
5.2 Google Drive 7
5.3 Azure DevOps 7
5.3.1 Scrum 8
5.3.2 Boards 9
5.3.3 Sprints 10
5.3.4 Activity Feed 11
5.3.5 Repos 11
5.3.6 Pipelines 11
5.3.7 Release 12
5.3.8 Deployment groups 13
5.3.9 Deployment Pools 13
6 Frontend 14
6.1 Oversigt 14
6.2 Funktionalitet 14
6.2.1 Administrative værktøjer 16
6.2.1.1 Indstillings panelet 16
6.2.1.2 Brugerpanelet 17
6.2.1.3 Logpanelet 18
6.3 Design (UI/UX) 18
6.4 API Brug 19
6.5 Styling 19
6.6 Program struktur 20
6.7 React 20
6.7.1 Hvad er React? 20
6.7.2 Hvorfor React? 20
6.7.3 Opsætning 21
6.7.4 Login og verificering 22

6.7.5 Komponenter 23
6.7.6 Data Flow og State Management 26
6.7.7 Unit & Integration Test 26

7 Domains & VPS 28
7.1 Anskaffelse & Windows installation 28
7.2 IIS 29
7.3 HTTPS 31

8 SQL 33
8.1 Installering 33
8.2 Tabeller og kolonner 34
8.3 Primary Keys 34
8.4 Foreign Keys 35
8.5 Typer 35

9 Backend 36
9.1 Models 36
9.2 API (Application Programmable Interface) 37
9.3 SQL Database access 38
9.4 Hashing/Salting Passwords 38
9.5 Scheduled Tasks 39
9.6 CORS (Cross-origin resource sharing) 40
9.7 Asynkronicitet 40
9.8 Authentication / API Key 41
9.9 Ip Rate Limiting 42
9.10 Mailservices 42
9.11 Tests 43

10 Fremtid/afgrænsning 45
10.1 Funktionelle krav 45

11 Konklusion 46
11.1 Gruppearbejde 46
11.2 Projektstruktur 46
11.3 Idéen 46
11.4 Opsummering 46

1 Hvem er vi?

1.1 Frederik Christensen

The project guy!
Har arbejdet i 3 år som teknisk konsulent i D4. Han har ydermere skrevet kontrakt efter endt skoleforløb med D4 som projektleder. I sin tid hos D4 har Frederik beskæftiget sig med følgende:

- SQL.
- Kodet windows applikationer i C#.
- Redigeret kodefiler.
- Opsætning af servere.
- Diverse projekter. Projekthåndtering.

Derudover har Frederik arbejdet hårdt på at blive bedre til "far jokes", og et større gennembrud forventes ultimo december/primo januar.

1.2 Casper Hilligsøe

The pretty designer!
Casper er uddannet grafisk designer og har en passion for at lave spil i Unity. Når han ikke arbejder på sit spil, kører han gerne en tur på cyklen. Han kører som en brækket arm. To gange. Han er en person der er dedikeret og slipper ikke tingene før de er gjort færdige. Primære områder:

- C#, C, C++, HLSL[1], JavaScript m.m.
- Spil-programmering.
- Grafisk design.
- Frontend.

1.3 Henrik Vincents Poulsen

The hardcore coder!
Er en alt mulig mand og er god til løse udfordringer. Når den korte frustrations periode efter noget er gået galt er ovre, iler han til at finde en løsning, der ofte kan mere end hvad der var den originale idé. Henrik er også en "oldschool guy" der har taget far jokes til sig, samt bruger frasen "Dengang jeg var ung" en gang imellem. Når Henrik ikke leger med sine unger, hygger han sig med:

- C#, JAVA, C++, SQL, XML m.m.
- Android Apps
- Server Backend
- Unity spildesign
- Systemkonfigurationer

[1] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

2 INDLEDNING

Implementeringen af et online vaskeri-bookingsystem kan gøre det nemmere for boligforeninger at administrere deres vaskerier, da en beboer selv kan registrere en fejl på maskinerne, samt hurtigt få en status. Det vurderes, at indførelsen af et online bookingsystem, vil reducere antallet af fejl fra beboerne, samt give en hurtigere reaktionstid på de fejl ved maskinerne i vaskeriet meldt af brugeren.

Der findes allerede digitale vaskerisystemer, som prøver at løse dette problem. Disse løsninger er meget dyre, da det er et komplet system med nøglekort til maskinerne. Dette er dog ikke aktuelt for mindre boligforeninger. Det er vores hensigt at lave en billigere løsning, der ikke er afhængig af fysiske begrænsninger, såsom automatiserede maskiner og nøglekort.

3 PROBLEMFORMULERING

Mange vaskeriers nuværende løsning på vasketids-reservering, er enten en tavle eller en logbog, hvori beboerne skriver deres ønskede tider ind. Beboerne skal gå fysisk ned for at se, om der er ledige tider. Man oplever endvidere, at folk ikke husker deres reserverede tider, hvilket resulterer i utilfredse beboere.

Hvis der er maskiner der er gået i stykker, skal beboerne finde kontaktinformationer på vicevært eller boligforening, for derefter at ringe og melde maskinerne i stykker. Herefter skal viceværten tjekke om den stadig er i stykker, for derefter at kontakte en tekniker. Denne fremgangsmåde er tidskrævende og der kan derfor gå lang tid før maskinerne bliver repareret.

Alternativt kan boligforeningerne have tilkøbt et integreret digitalt vaskesystem, som kan være mere end de har brug for. Disse løsninger er ofte meget dyre og er derfor ikke til interesse for boligforeningerne.

På baggrund af ovenstående har vi derfor valgt at fokusere på følgende:

1. **Hvordan laver man et konkurrencedygtigt cross-platform system, som er nemt og hurtigt at bruge for både administratorer og brugere?**

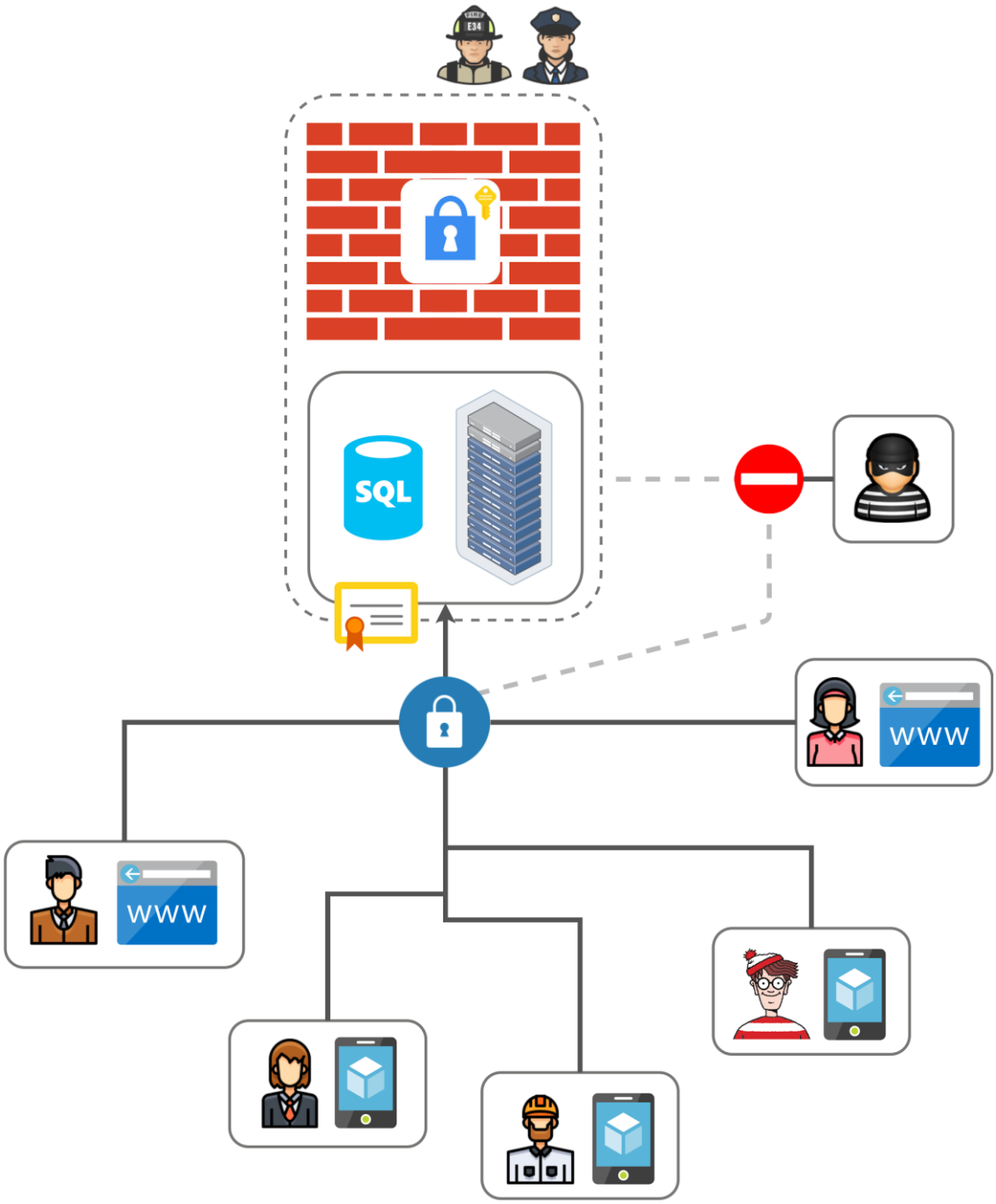
---

2. **Hvordan udvikler man systemet så det er sikkert med mange simultane brugere og med høj driftssikkerhed?**

4 PROJEKTBESKRIVELSE

Vi vil lave en løsning, som er billig, både i drift og indkøb og samtidig bibeholder de vigtigste funktioner i et moderne vaskesystem. Løsningen bliver en webapplikation med tilhørende app, hvorpå beboere kan logge sig ind og reservere tider, samt få notifikationer ud på e-mail.

Vi har sat en Windows Server 2019 op med en MSSQL database og en .NET Core 3.1 server som serverer data fra database via API til en frontend React hjemmeside.


SQL
WWW
WWW

5 Projektstyring

I ethvert projekt skal der være en form for projektstyring. I dette afsnit har vi valgt at fokusere på følgende anvendte værktøjer:

- Discord
- Google drive
- Azure DevOps

5.1 Discord

Discord[2] er anvendt som platform for den daglige kommunikation.
Vi har afholdt statusmøder alle hverdage kl. 09 og 14, for at skabe konsensus om dagsplanen, samt have mulighed for at drøfte eventuelle udfordringer og spørgsmål. Det er muligt på Discord at dele sin skærm, der gør fælles problemløsning en del nemmere.

5.2 Google Drive

Google Drive er benyttet som lager for bl.a. vores dokumenter.
Vi har tilmed brugt Google Docs fremfor Word, da Docs kan anvendes skriftligt af alle på samme tid. Dette har været effektivt ifm. debat om hvordan vores rapport, kravspecifikation m.v. skulle udformes. Google Docs bliver opbevaret i Google Drive.

5.3 Azure DevOps

Azure DevOps har været et essentielt redskab i vores projektstyring.
Det første der skal gøres ved brug af Azure DevOps (også kaldet “Devops”), er at oprette sin organisation. Herunder kan oprettes et eller flere projekter. For at der er flere der kan arbejde på projekterne, skal der tilføjes personer til organisationen. Når personer er oprettet i organisationen, kan de placeres i de respektive projekter. Tilmed kan disse personer opdeles i teams på projekterne. Personerne kan tildeles forskellige rettigheder på organisationsniveau, men også separat på projekterne.
Når man har oprettet et projekt, har man som standard seks moduler til rådighed.
Hvert modul har sin egen kompetence - men alle med samme formål. DevOps er nemlig skabt som et værktøj til agil projekthåndtering. Når et nyt projekt oprettes i DevOps, er det muligt at vælge hvilke processer der ønskes benyttet til projekthåndtering. Vi har valgt DevOps standardløsning inden for proces (kaldet ”Basic”). Den inkorporerer enkelte Scrum elementer, men muliggør strukturering efter eget ønske.

[2] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

All processes

Processes Fields

Name	Description
Basic (default)	This template is flexible for any process and great for teams getting started with Azure DevOps.
Agile	This template is flexible and will work great for most teams using Agile planning methods, including those practicing Scrum.
Scrum	This template is for teams who follow the Scrum framework.
CMMI	This template is for more formal projects requiring a framework for process improvement and an auditable record of decisions.

5.3.1 Scrum

Scrum er en anerkendt metode, som benyttes inden for agil projekthåndtering. Scrum bruger diverse redskaber og principper til at gøre det så agilt som muligt. Metoden går meget op i planlægning og forberedelse. Dette gøres ved at afholde daglige statusmøder og planlægge i intervaller – også kaldet ”sprints”. I et sprint, som ofte forløber over 14 dage, indlægges en bunke af opgaver, der først er planlagt og derefter gennemgået. Dette bevirker at der altid er overblik og at koordinationen af projektet forløber nemmere.

Mange møder kan i sig selv være en negativ motivationsfaktor og mindske effektiviteten i arbejdet, da folk ikke ønsker flere møder end højst nødvendigt. Dog er Scrums møder designet til at være korte, med struktur og mening ved hvert punkt på dagsordenen.

I Scrum har man fastsat 3 roller:

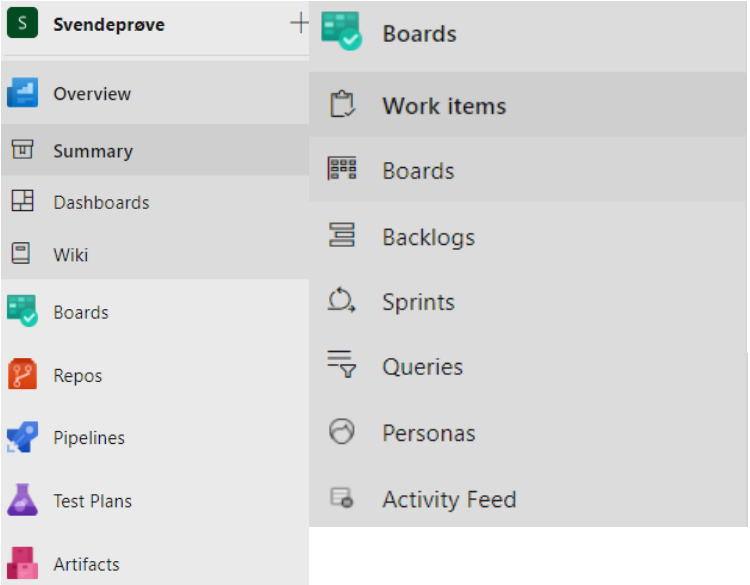
- Scrum Team
  - Det team som laver produktet. Det kan være udviklere, designere m.m.
- Product Owner
  - Er en person som skal sikre, at alle opgaver bliver udført som aftalt, sikre at alle godkendelser er på plads, samt tage stilling til de ikke-nåede opgaver.
- Scrum Master
  - Er en person der sikrer at effektiviteten er på sit højeste ved at holde styr på Scrum metoden. En Scrum Master er neutral og skal sikre sig at projekthåndteringen følger de aftalte Scrum metoder.

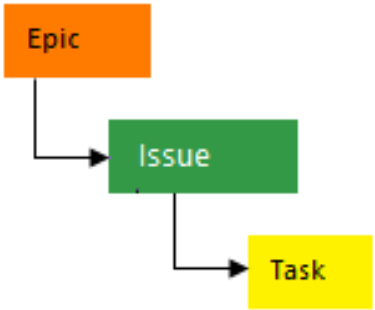
I praksis bruges ScrumBut af mange virksomheder og projektgrupper. ScrumBut er Scrum i sin egen version, hvor ikke alle elementer tages med fra Scrum. Det kan f.eks. være, at der er unødvendig planlægning eller andre principper fra Scrum, som ikke kan implementeres i projektet. DevOps kan siges at være ScrumBut. De har mange af de principper som Scrum har inkorporeret i softwaren. De har både Sprints og agile boards, samt mange af de back-log[3] funktioner, som Scrum har. Dette drager vores projekt også nytte af.

[3] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

5.3.2 Boards

Når et nyt projekt er oprettet lægges der ud med seks moduler (se nedenstående billede.[4]
Vi har primært haft gavn af ”Boards”, “Repos” og “Pipelines”.





Boards og dets undermoduler vil efter vores vurdering indgå i traditionel projektstyring.
Boards bruges som opgavehåndtering, hvor man har “work items”, “kanban board”, “Sprints” m.m.
Man begynder ved at oprette work items, som er kategoriseret i tre niveauer: “Epic”, ”Issue”, og ”Task” (førstnævnte er den største opgave (jf. billede)).
En god idé er at linke Issues med Epics, Task med Issues m.v. med det formål at skabe et overblik over hvilke opgaver der hører sammen. Dette giver samtidig et endnu bedre overblik, når man når til det næste submodul “boards”. boards ligger under hovedmodulet ”Boards”. For overskuelighedens skyld staver vi submodulet med lille b. boards er et digitalt kanban board, der kan lidt ekstra.

Work items

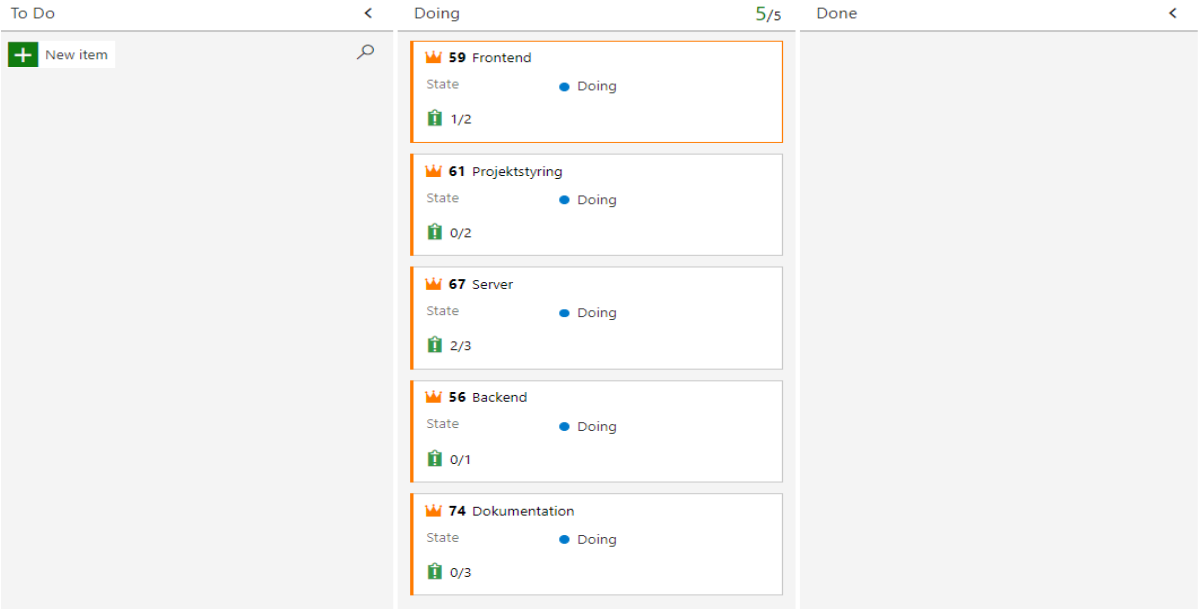
Recently updated | New Work Item | Open in Queries | Column Options | Import Work Items | Recycle Bin

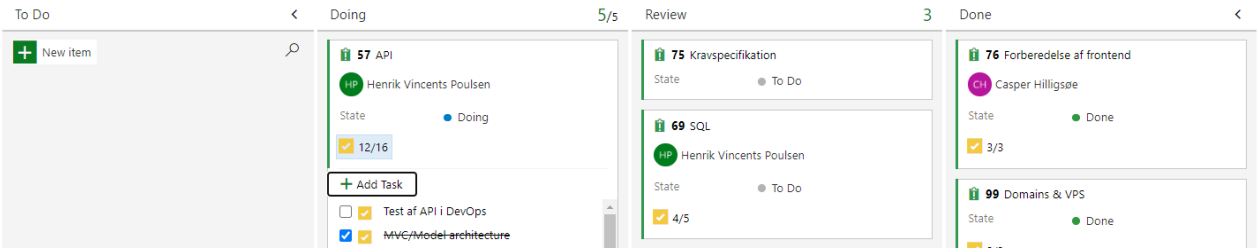
Filter by keyword | Types | Assigned to | States | Area

ID	Title	Work Item Type	Assigned To	Iteration Path
67	Server	Epic	Unassigned	Svendeprøve
61	Projektstyring	Epic	Unassigned	Svendeprøve
59	Frontend	Epic	Unassigned	Svendeprøve
57	API	Issue	Henrik Vincents Poulsen	Svendeprøve
103	Rapport	Issue	Unassigned	Svendeprøve
69	SQL	Issue	Henrik Vincents Poulsen	Svendeprøve
62	DevOps	Issue	Frederik Christensen	Svendeprøve
104	Persondata	Issue	Unassigned	Svendeprøve
63	Dagbog	Issue	Unassigned	Svendeprøve
75	Kravspecifikation	Issue	Unassigned	Svendeprøve
60	React	Issue	Unassigned	Svendeprøve
98	Online Documentation	Task	Henrik Vincents Poulsen	Svendeprøve\Sprint 3

[4] Se “Procesrapport Bilag” side 13 for at få et overblik over alle moduler og submoduler.

Jf. nedenstående billede kan man placere sine Epics og Issues på kanban boardet.

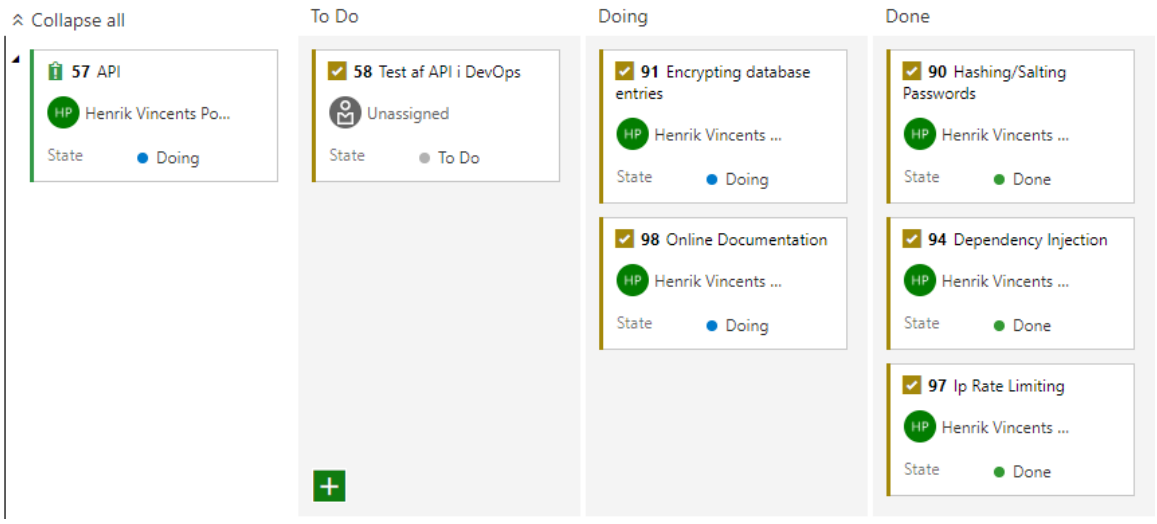




På billedet ses en oversigt over Epics og på det efterfølgende billede ses en oversigt over Issues. Når et Issue er "done" bliver det vist på den Epic, den er linket til. Vi har valgt at tilføje en ekstra kolonne ved Issues (kaldet "Review"), da vi har ting der skal gennemgås, gruppens medlemmer imellem, så alle har styr på hvad der er blevet lavet.

5.3.3 Sprints

Sprints er som tidligere nævnt opgaveintervaller, hvor man placerer en bunke af opgaver der skal laves inden for en tidsramme – oftest på 14 dage.
Vi har valgt at vores Sprints varer syv dage over fire uger. I vores forløb har Sprints ikke været det vigtigste værktøj, da det oftest henvender sig til større og længerevarende projekter. Dog har vi forsøgt at anvende og følge det.



5.3.4 Activity Feed

Activity Feed er en log over hvad der er blevet ændret på DevOps, som kan være brugbar f.eks.i tilfælde af fejl.
Det er ikke noget der ofte bliver brugt, men kan være afgørende for at kunne se eventuelle fejltagelser der er sket og dermed kunne udbedre fejlen(e).

5.3.5 Repos

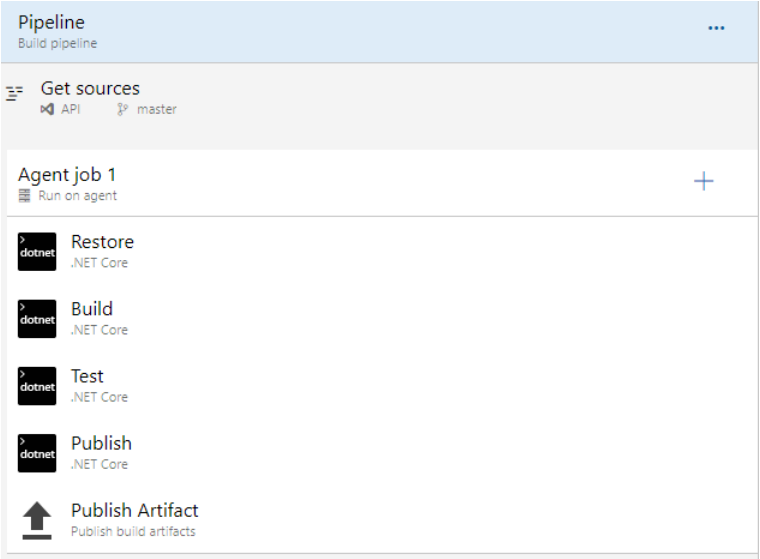
Repos er Azure's svar på GitHub[5]. Selvom DevOps har gjort GitHub tilgængelig til deres moduler, så det er muligt at importere og eksportere derfra, så har Azure deres egen opbevaring af filerne. Udover at kunne se filerne, kan man også se "commits", "pushes", "branches" og "pull requests". Dette er alle velkendte funktioner fra Git[6], GitHub og Visual Studio.

5.3.6 Pipelines

Pipelines har været en stor gevinst for vores projekt. Pipelines skal ses som enhver anden kode man skriver. Man har muligheden for at sætte sin pipeline op via YAML til at gøre forskellige ting. YAML er en filtype der ofte bliver brugt til konfigurationsfiler. Vi har dog valgt at bruge et indbygget værktøj fra Azure til at opsætte vores pipeline, da den imødekom vores krav. Man kan se en eksport af vores YAML kode i vores bilag.[7]

[5] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[6] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[7] Se YAML kode i "Procesrapport Bilag" på side 14 og 15.

Vores pipeline i DevOps bruges til at"restore", "build", "test" og "publish".



Restore bruges til at tilføjelser/NuGet pakker bliver downloadet af Azure. Dermed kommer der ikke fejl når Azure skal bygge en solution[8], som bruger disse tilføjelser.

Build tester om projektet kan "builde", som det ville gøres i Visual Studio. Der gives en fejlmeddelelse, hvis dette ikke er muligt.

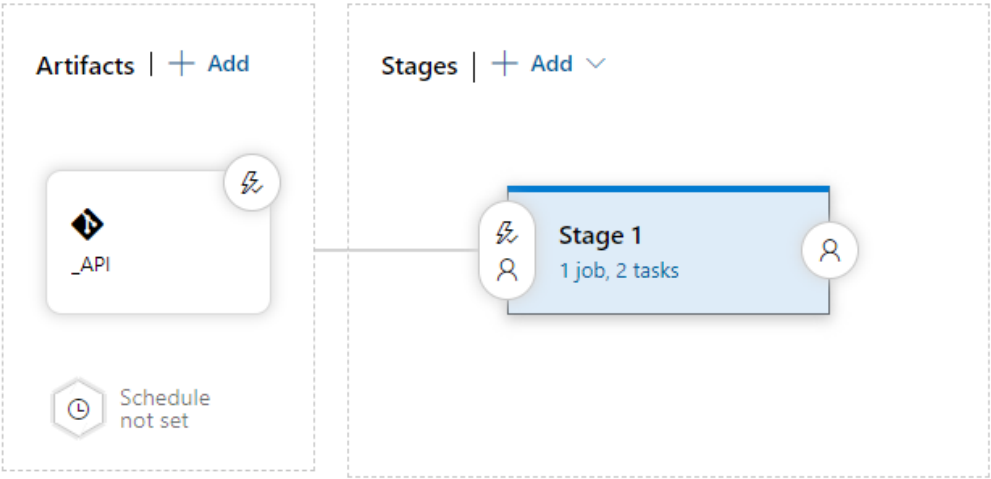
Test har vi ikke implementeret i vores pipeline, men det kan sættes op til at dække kodefejl, forbindelse til SQL m.m. Dette har ikke været en høj prioritet og vi har derfor ikke nået det.

Publish betyder at Azure nu har udgivet det til deres egne servere og at alt er gået igennem. Hvis en af disse trin i pipeline fejler, vil processen ophøre og der gives en fejlbesked.

5.3.7 Release

Her sættes Azure op til at release de "builds" vores pipeline har gennemført og "publish" det til vores server. Dette vil sige, at hver gang vi sender ny kode til DevOps, vil Azure gennemgå koden for at tjekke at det virker, for derefter at "publish" det på serveren, hvis der ingen fejl er. Dette kan umiddelbart lyde risikabelt, da det hele gøres i et step og det kan sendes ud på store produktionsservere ved et tryk, men der kan anvendes filtre og restriktioner, samt utallige test på pipelinen, der gør det sikkert.
Nedenstående billede viser, at den skal tage vores "build", som har været igennem pipeline og lave en "deployment". Deployment delen er det "lille lyn" i højre top af Artifacts billedet.
Under "Stages" er opsætningen af deployment, hvor vi har peget på vores IIS server.

[8] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

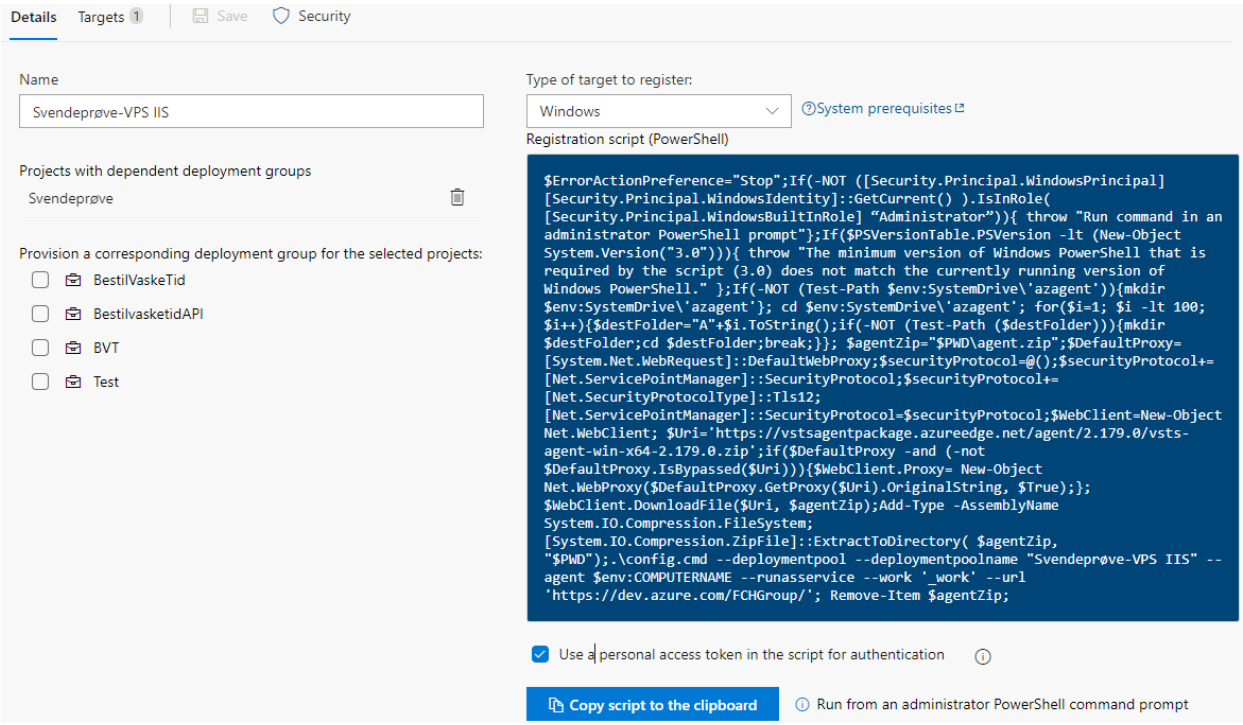


5.3.8 Deployment groups

Deployment groups er en gruppe af agenter[9], som udfører opgaver for Azure. De bruges f.eks. til at køre de pipelinekommandoer, som skal anvendes for at bruge en pipeline.
Det er samtidig også en gruppe af maskiner, som man selv peger på. I vores projekt har vi kun peget på en enkelt, som er vores IIS server.

5.3.9 Deployment Pools

Deployment pools er stedet hvorpå opsætningen til serveren sker og sørger for at agenterne har de nødvendige rettigheder til at "publish" det til serveren.
Jf. nedenstående billede skal man køre et powershell script med en nøgle, der giver rettighederne.



[9] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

6 Frontend

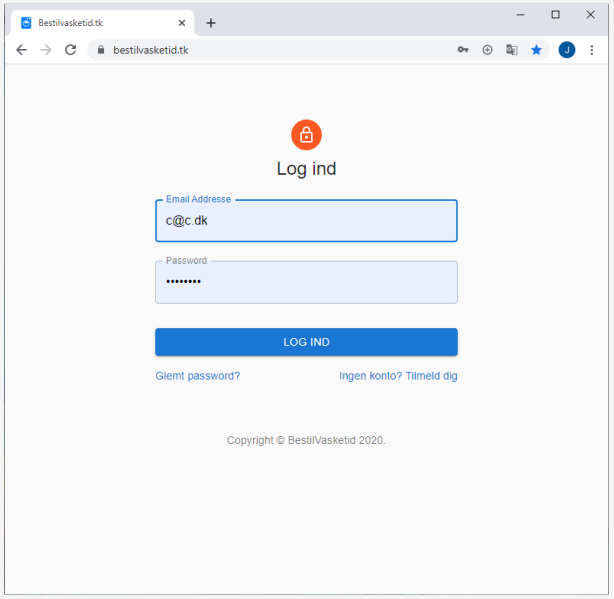
6.1 Oversigt

Projektets mål har været at udarbejde en hjemmeside og tilsvarende app som muliggør visning og bestilling af vasketider samt administration af samme.
Det har været nødvendigt at afgrænse vores frontend til primært at omhandle vores kunder og deres brugere/administratorer. Siden er således ikke ment som et værktøj for FCH Group til at oprette nye boligforeninger, kunder osv.

6.2 Funktionalitet

Det første som møder brugeren er **login** siden. Herfra er det muligt at:

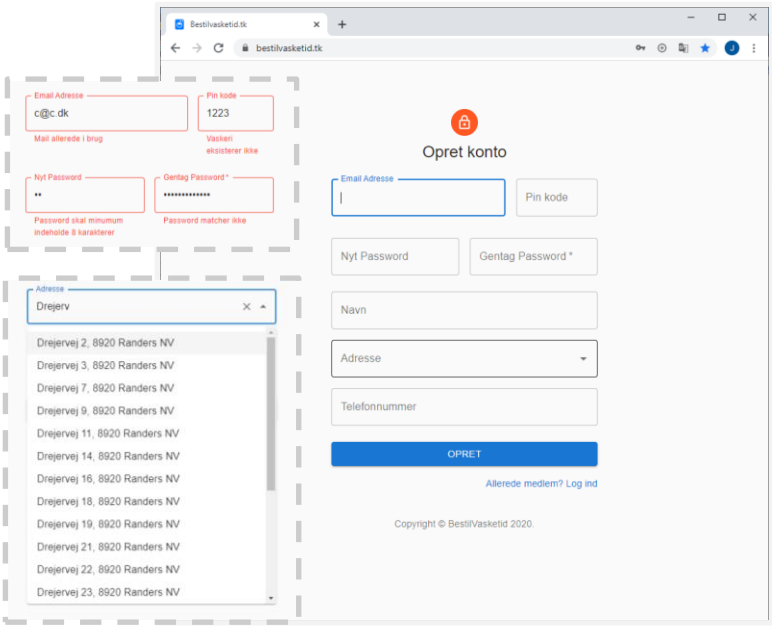
- Fortsætte til vaskeri via login.
- Gå til "opret bruger".
- Gå til "glemt bruger" og genoprette password via email.



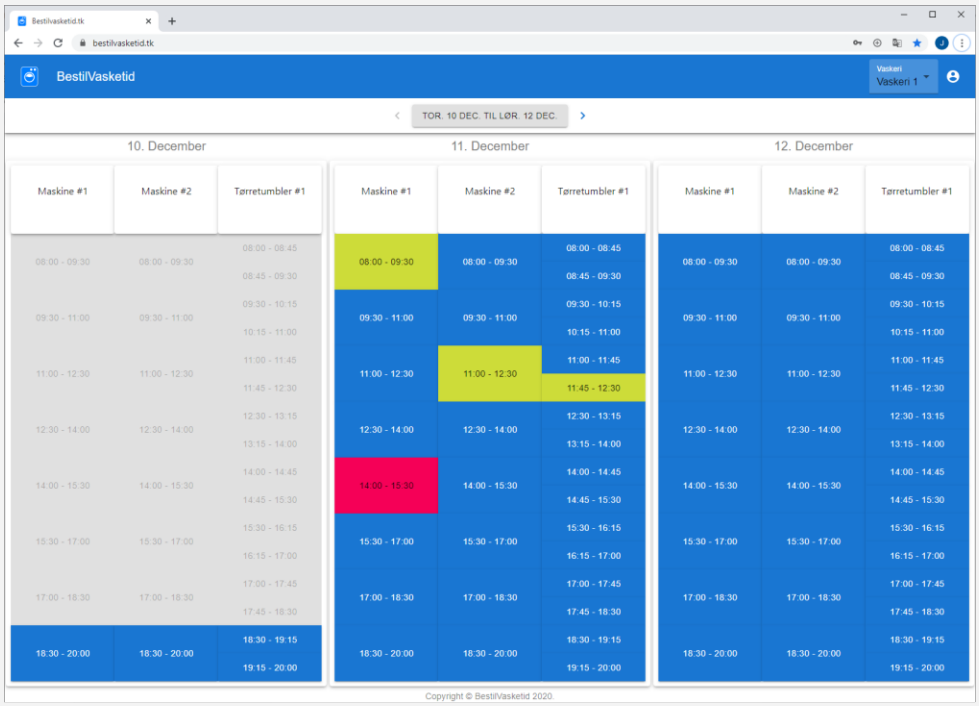
Ved login bliver brugerens e-mail og password valideret via vores API. Hvis valideringen er succesfuld, returnerer API'et brugerens data - som efterfølgende sendes til vaskeriets forside.

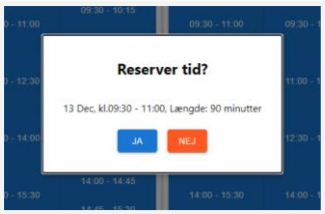
Vi har lagt stor vægt på validering af brugerinput, fra formular udfyldning til knapper. Fejl skal være nemme at se og ikke mulige at gennemføre.

Vi gør eksempelvis brug af Dawa's adresse API til auto udfyldning af formularfelt, når kunden indtaster deres adresse (fuzzy search[10]). Dette gør det nemt for brugeren, men sikre også adressen er valid.

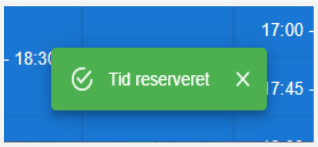


[10] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"



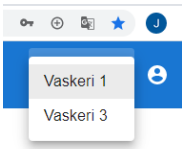


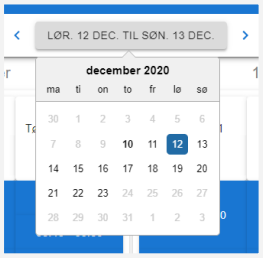
Når brugeren er logget ind, ankommer de til **bestillingssiden**. Her kan man se, bestille og slette vasketider. Siden er responsiv og skalerer automatisk baseret på skærmstørrelse og antal elementer. Ved valg af tid promptes brugeren med en modalboks[11] (t.v) og en popup (t.h) bekræfter eventuel reservation.



Vaskeriets data hentes via Bestilvasketids API, og tidernes farver indikerer deres status i forhold til den aktive bruger.

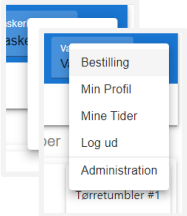
Hvis kunden har adgang til flere vaskerier vil en “dropdown” være synlig og muliggøre at skifte imellem disse (t.h).





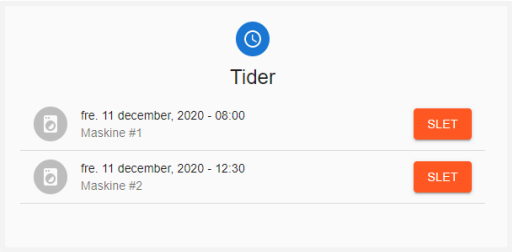
Skemaets dato kan ændres med pilene eller via dropdown (t.v).

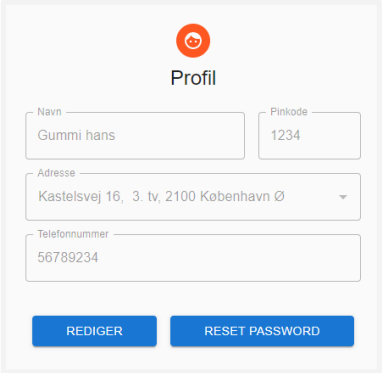
Et ekstra menupunkt, kun synlig for administratorer, giver adgang til vaskeriets administrative værktøjer (t.h).



[11] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

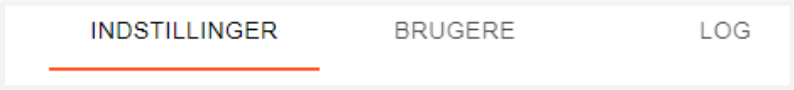
Brugerens profil samt tidsoversigt kan tilgås via profil ikonet.





6.2.1 ADMINISTRATIVE VÆRKTØJER

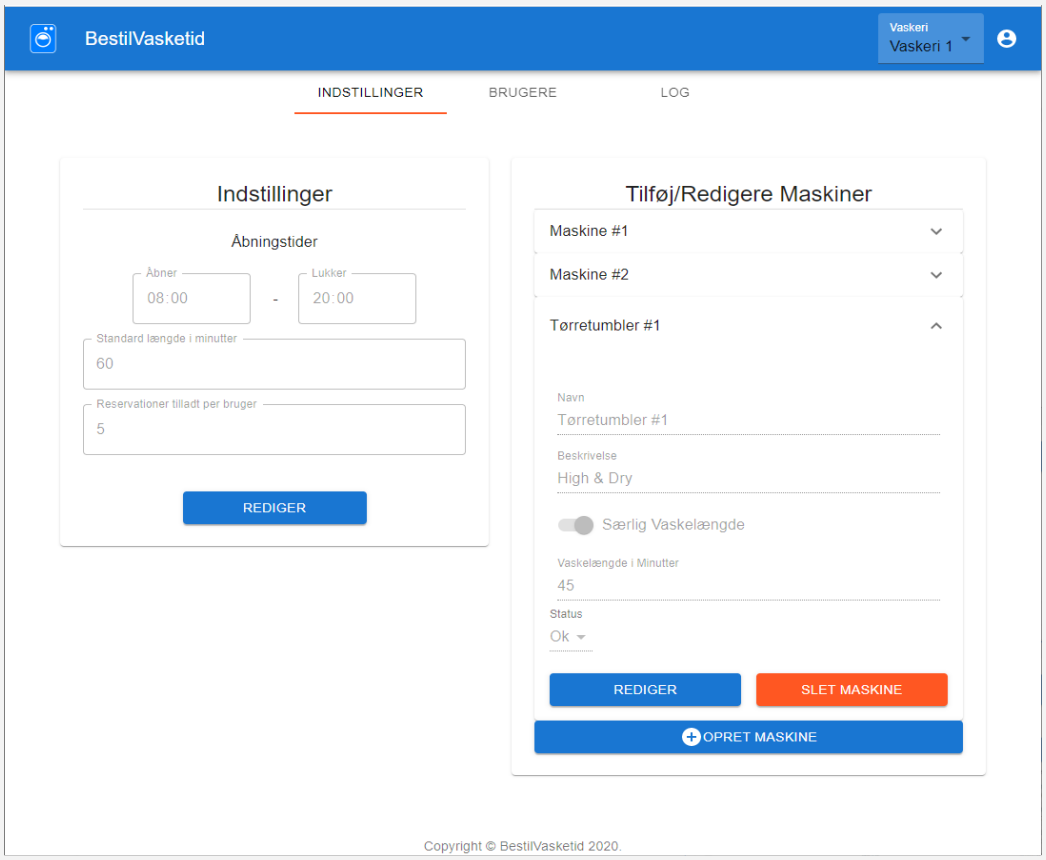
Administratorer kan tilgå vaskeriets **Administrative værktøjer.**
Siderne vises konditionelt[12] og styres gennem Værktøjspanelet øverst på siden.



6.2.1.1 Indstillings panelet

Giver mulighed for at ændre vaskeriets opsætning, tider samt tilføje/redigerer maskiner.

[12] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

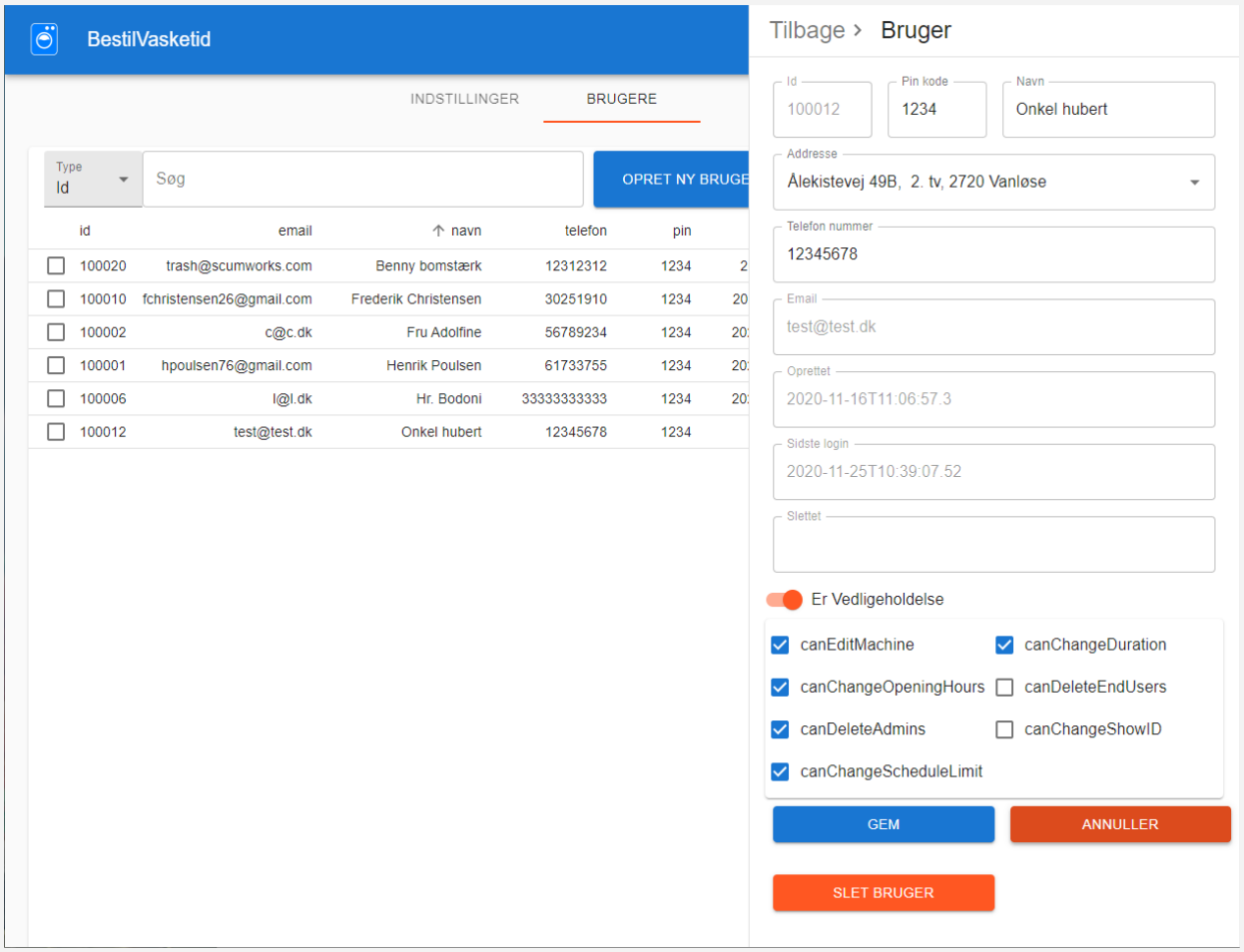


6.2.1.2

Brugerpanelet

Viser vaskeriets brugere (på samme PIN kode). Det er muligt at søge (navn, mail datoer osv), slette, redigere og oprette nye brugere.

Ved oprettelse af ny bruger, vil denne oprettes med et midlertidig tilfældigt password. Brugeren sendes efterfølgende en e-mail med link til at påbegynde en “password reset” procedure. Brugeren modtager en e-mail, som leder ind til en side hvor der kan indtastes et nyt password.

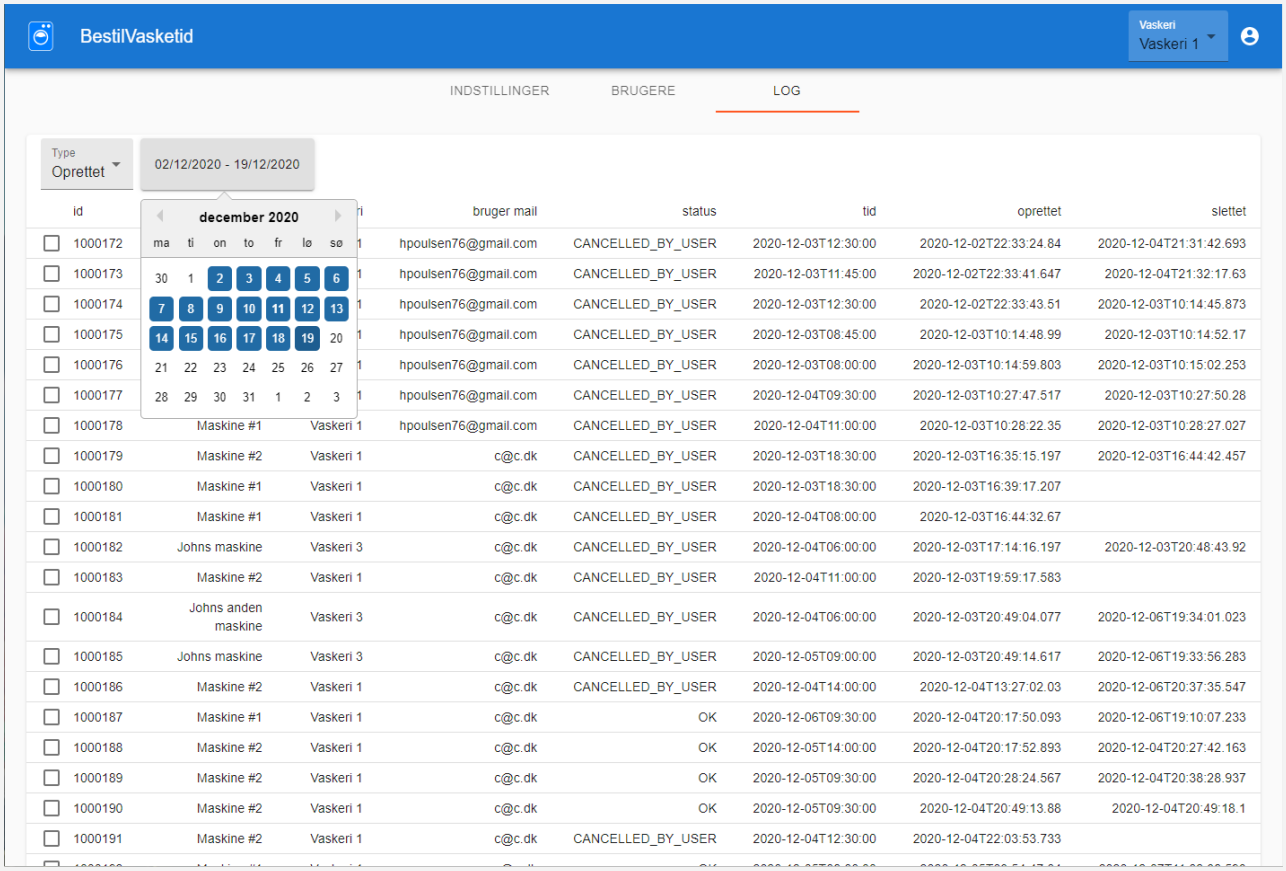


Når man klikker på brugerlisten, åbner brugerens fulde information op i en “side drawer[13]”. Administratorer med fulde rettigheder vil kunne tildele og redigere brugerens administrative rettigheder.
Tilladte handlinger, knapper og tekstfelter bliver alle dynamisk ændret ud fra administratorens egne rettigheder, altså personen som er logget ind.

[13] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

6.2.1.3 Logpanelet

Viser bestilte tider. Hvem, hvad og hvornår. Dette kunne eksempelvis give indblik i misbrug af vaskeri, samt muligheden for at slette unødige reservationer.



6.3 Design (UI/UX)

Brugerfladen er kundernes indgangsvinkel til vores produkt. Det er derfor vigtigt det giver en følelse af kvalitet og design, uden at gå på kompromis med funktionalitet og enkelthed. Bestilvasketid handler (som navnet antyder) først og fremmest om at brugere skal kunne bestille/afbestille vasketider samt se ledige tider. Vi har haft fokus på at gøre dette så simpelt og intuitivt som muligt.

Grundfarver		Varianter	
■ #000000	Sort	■ #cddc39	Grøn
■ #000000	Hvid	■ #f50057	Rød/bordeaux

Designet (UI[14]) er således skabt omkring:

- Responsivitet[15], UI'et skal vises korrekt uanset hvilken platform siden tilgås fra.
- Stilrent farvevalg samt simpel og overskuelig komposition af elementer.
- Stærke, klare farver som adskiller sig fra hinanden - hvilket gør siden mere læsbar for personer f.eks. med dårligt syn.
- Farve signalering, elementer som adskiller sig fra de tre angivne grundfarver, signalerer information og ikke bare designæstetik, f.eks. tider optaget, ude af brug eller reserveret af kunden selv). Opretholdelse af denne signalering giver et intuitiv overblik.

User experience (UX[16]) er fokuseret på:

- Enkel at bruge - kræver ikke prædisponeret kendskab til lignende sider eller applikationer.
- Adgang til de mest benyttede funktioner på forsiden, uden at skulle grave i menuer. Andre funktioner kan med få klik tilgås via vores dropdown menu.
- Feedback. Brugeren bliver informeret om resultatet af deres handlinger, oftest ved brug af en popup "snackbar/toast[17]".

6.4 API Brug

Bestilvasketid gør selvsagt brug af vores eget API til alt fra verificering af brugere, oprettelse af tider osv.
Al kommunikation foregår asynkront og det er således muligt at modtage og hente information, uden at siden låser eller hænger.
Bestilvasketid gør endvidere brug af adresse API'et Dawa (Danmarks Adressers Web API). Dette har gjort det muligt at auto fuldføre indtastning af adresser (fuzzy search), hvilket både hjælper brugeren, samt sikre mod fejlindtastet adresser.

```
export const APIPostCall = async (url, body) =>
{
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'Content-Type': 'application/json',
      'apiKey': 'SecretApiKey'

    },
    body: JSON.stringify(body)
  })

  return response
}
```

6.5 Styling

Bestilvasketid gør brug af UI frameworket Material UI[18] til at style de mange forskellige elementer siden består af.

[14] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[15] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[16] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[17] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[18] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

Frameworket gør brug af en "CSS-in-JS" stil, hvor stylingen indsættes direkte i komponenten. Resultatet er et komponent som ikke afhænger af eksterne stylesheets.

6.6 Program struktur

Flowchart diagram i **bilag**[19].
Kontekst/Dataflow diagram i **bilag**[20].

6.7 React

6.7.1 Hvad er React?

ReactJS er et opensource (MIT[21]), JavaScript baseret frontend bibliotek med henblik på opbygning af responsivt UI og interaktive hjemmesider. Det er lavet af Facebook som også vedligeholder kodebasen i samarbejde med virksomheder og brugere.

Ulig produkter såsom Angular er ReactJS ikke et altomfattende framework, men snarere en komponentbaseret løsning hvor moduler kan importeres og tages i brug ud fra projektets behov. Dette giver en vis fleksibilitet, ikke kun i struktureringen af programmet - men også i henhold til features som inkluderes.

6.7.2 Hvorfor React?

Målet har været at skabe en hjemmeside som både er responsiv og kan håndtere mange interaktive elementer uden at blive langsommere.

Traditionelt fungerer hjemmesider som MPAs[22] (multiple page application), hvor serveren skal kontaktes når f.eks. brugeren anmoder om at se en ny side. Denne transaktion er relativ langsom, både påvirket af hvor stresset webserveren er, samt brugerens internethastighed.
Reacts SPA[23] (single page application) muligører at processen "frontloades[24]", hvilket i Bestilvasketids omfang, betyder at webserveren kun kontaktes èn gang - alle UI ændringer vil herefter blive udført på klientens side. Dette reducerer stress på webserveren, som generelt er bedre til at håndtere større enkelte dataoverførsler end mange små.

[19] Se "Flowchart diagram" i "Procesrapport bilag" side 4 og 5.
[20] Se "Kontekst/dataflow diagram" i "Procesrapport bilag" side 5
[21] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[22] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[23] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[24] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

SPA konceptet som vi benytter har endvidere gjort det nemmere at oprette hjemmesiden som en PWA (Progressive web app, se evt. **guide i bilag**[25])), da klienten nemt kan operere i offline tilstand. Der skal dog stadig være webadgang for at se og bestille tider. Resultatet er en hjemmeside som fungerer både i normale browsere samt installeret som app (f.eks. på telefonen).
Fordelene ved at udvikle til flere platforme fra èn enkelt kodebase er enorme og fjerner den arbejdsbyrde der eksempelvis ville findes i sideløbende apps (Android, swift osv.) - for slet ikke at nævne vedligeholdelse samt konsolidering af features imellem.

En anden af Reacts funktioner er dens tilgang til DOM [26](Document object model) manipulation via en såkaldt "Virtuel DOM[27]". Browserens DOM er konstrueret som et data træ, som repræsenterer de forskellige komponenter på siden. Opdatering af dette træ er hurtigt, men specielt den efterfølgende genskabelses (render) og linking af de forskellige elementer er langsomt - noget som kun bliver værre som siden/træets grene gror.

Hjemmesider som Bestilvasketid er ofte særligt ramt af disse problematikker, da et stort antal over- og under elementer benyttes for at fremvise sidens data, f.eks. bestillingsoversigten, og logs.

React optimerer denne process ved at oprette en kopi af DOMens træ. Når ændringer sker, oprettes et nyt data træ, som herefter tjekker differencerne mod kopien. Dette gør det muligt at finde forskellene og derefter kun opdatere påvirkede elementer i den originale DOM.

Fordele

- **Virtual DOM** sikrer en mere effektiv DOM manipulation.
- **Automatisk håndtering af UI state**[28]. React opdaterer komponenter når state ændres.
- **Generiske komponenter** som kan genbruges i og uden for projektet.
- **API paritet** mellem ReactJS og React Native (og ReactVR)
- **Asynkron**[29] **state håndtering**.
- **Dokumentation og fællesskab**, stor vidensdeling på internettet.

Ulemper

- **Eksterne pakker/modul basering,** flere konflikter end et "all-in-one" framework.
- **"Separation of concern**[30]**"** blanding af HTML og JavaScript via JSX gør oftest UI "tightly coupled[31]" med logik.

[25] Se "React til PWA (Progressive Web App)" i "Procesrapport bilag" side 8, 9, 10, 11 og 12
[26] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[27] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[28] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[29] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[30] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[31] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

6.7.3 OPSÆTNING

React er et modulært system og det er op til den enkelte bruger at vurdere, hvor meget eller lidt man ønsker at bruge af det.
Et React projekt gør oftest brug af 3 primære ting, en package manager, bundler og kompiler.

Package manager
Bestilvasketid bruger npm[32] (Node package manager) til håndtering (installering, opdatering og bogføring) af dependencies/moduler. Dette udgør at vi bl.a. kan udelade node_modules mappen fra repos (kan indeholde flere hundrede tusinde filer) og i stedet downloade disse filer separat.

Bundler
Bundlerens job er at samle/linke de forskellige filer og dependencies, for til sidste at generere - i samarbejde med kompileren - et eller flere færdige bundles som browseren kan bruge. Kompileringen af disse bundles kan variere, afhængig af om der er tale om "development" eller "production" builds. Sidstnævnte optimeres oftest med minification[33].

Bestilvasketid gør brug af Webpack[34] som går godt i spænd med vores kompiler.

Kompiler
Browsere kan ikke forstå React kode og understøtter ikke altid de seneste JavaScript funktioner som benyttes i projektet. Det er derfor nødvendigt at kompilere til almen JavaScript/HTML som supporteres og kan vises i de fleste browsere.

Bestilvasketid benytter Babel[35] til at kompilere,som er et populært valg eftersom det understøtter Reacts JSX[36] sprog.

[32] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[33] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[34] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[35] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[36] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

6.7.4 LOGIN OG VERIFICERING

Når brugere tilgår vores hjemmeside, vil de først og fremmest dirigeres af vores routes, som har til opgave at læse url'en og route herefter.

```
<MemoryRouter >
  <div>
    <Switch>
      <Route path="/" component={(props) =>
      {
        if (window.location.href.includes('?resetpassword'))
          return <ResetPassword {...props} />
        else
          return <SignIn {...props} />
      }} exact />
      <Route path="/signup" component={SignUp} exact />
      <Route path="/forgotpassword" component={ForgotPassword} exact />
      <Route path="/resetpassword" component={ResetPassword} exact />
      <ProtectedRoute exact path="/app" component={Home} /> */
    </Switch>
  </div>
</MemoryRouter>
```

Dette gør det muligt at tilgå forskellige sider (f.eks. opret ny bruger) via url adresse samt at checke for query strings[37] i URL'en. Disse query strings benyttes eksempelvis når brugeren genopretter sit password via e-mail.

Ved password reset modtager brugeren en url som indeholder vores reset tag samt

Hej lore. Du har anmodet om at nulstille dit kodeord.
Klik på linket for at ændre kodeordet
https://bestilvasketid.tk/?resetpassword#25131ff266947ef3

[37] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

For at undgå brugere kan route til Bestilvasketids forside uden at være verificeret/logget ind - benyttes komponentet **<ProtectedRoute>** til at checke status af vores **Auth.js** klasse. Er brugeren ikke godkendt, omdirigeres der til sidens rod "/".

Denne godkendelse sker i **SignIn.js** hvor formular-data sendes til Bestilvasketids API, som herefter returnerer en statuskode.

```
if (auth.isAuthenticated()) {
  return <Component {...props} />;
} else {
  return (
    <Redirect
      to={{
        pathname: "/",
        state: {
          from: props.location
        }
      }}
    />
  );
}
```

Checker om

Ved login sendes form-data til

```
const onSubmit = props => form =>
{
    // Authenticates login.
    auth.login(setLoginError, (form), (data) =>
    {
        props.history.push({
            pathname: "/app",
            search: '?home',
            state: data
        });
    });
}
```

Auth.login checker bruger via API, sætter "isAuthendicated" baseret på statuskode returneret, og hvis "OK/200" kalder callback funktionen med brugerens data.

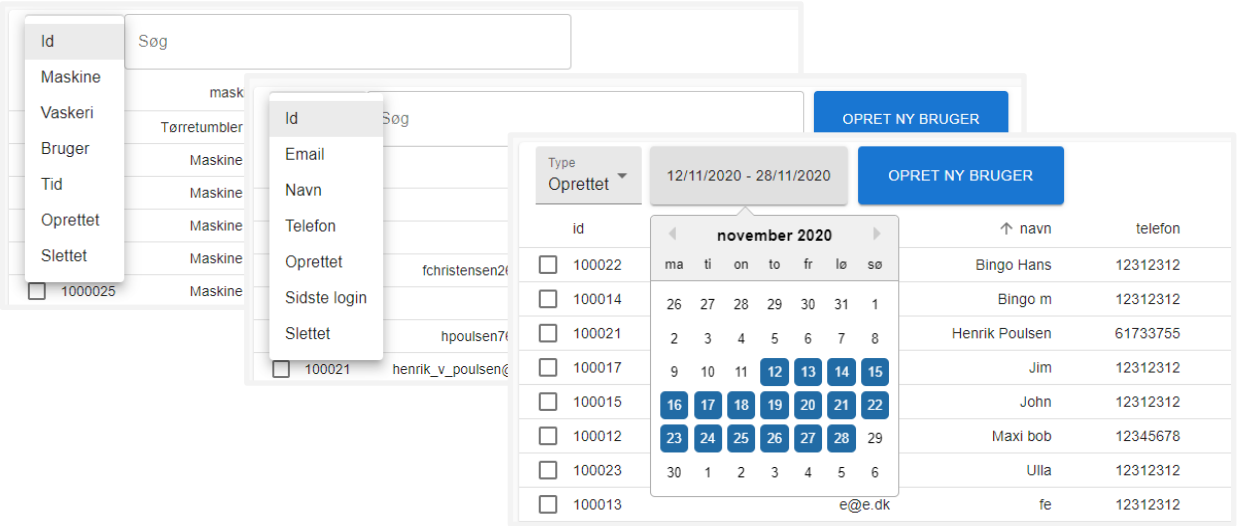
```
login = async (setLoginError, form, event) =>
{
    const response = await APIPostCall(
        'https://bestilvasketid.ml/api/user/login',
        { 'email': form.email, 'password': form.password }
    )

    // 401 - mail exists, password wrong. 404 - mail not found
    if (response.status == 200)
    {
        this.authenticated = true;

        if (event != undefined)
        {
            event({ ...await response.json() });
        }
    }
    else
    {
        setLoginError(true)
        this.authenticated = false
    }
    return response.status
}
```

6.7.5 Komponenter

Komponenter er Reacts byggesten. Det er måden vi strukturerer vores UI layout og funktionalitet. Komponenter er enten klasser eller funktioner (såkaldte funktionelle komponenter) som returnerer JSX (JavaScript XML) kode. Disse komponenter vil som oftest indeholde deres egen logik, funktioner og variabler - men kan endvidere modtage disse via properties (props) eller

context providers[38]. Målet er at skabe genbrugelige, selvforsynende UI elementer som kan fungere uafhængigt fra det resterende projekt.
For Bestilvasketid har dette koncept vist sig specielt nyttigt til at genere de mange UI elementer, ud fra et generisk grundlag. Alt fra “modal popups” til søgeredskaber er konfigurerbare og selvforsynende, hvilket gør dem nemme at genbruge - i og uden for projektet.

Eksempelvis vores søge redskabs funktionalitet i log og



38

```
<Searchbar
  selected={selected}
  setSelected={setSelected}
  setRows={setRows}
  numSelected={selected.length}
  content={schedules}
  massDeleteButton={
    {
      buttonText: 'Slet',
      buttonColor: 'secondary',
      modalTitle: 'Slet valgte reservationer?',
      modalButton1Text: 'Slet',
      modalButton1Function: (async (e) => { return await APICallDeleteSchedule(e, context, false)}),
      modalButton2Text: 'Annuller',
    }
  }
  searchPrefs={
    [
      { name: 'Id', property: 'id', filter: Global.searchFilter.CONTAINS },
      { name: 'Maskine', property: 'machineName', filter: Global.searchFilter.CONTAINS_CASEINSENSITIVE },
      { name: 'Vaskeri', property: 'laundryName', filter: Global.searchFilter.CONTAINS_CASEINSENSITIVE },
      { name: 'Bruger', property: 'email', filter: Global.searchFilter.CONTAINS_CASEINSENSITIVE },
      { name: 'Tid', property: 'time', filter: Global.searchFilter.DATE_RANGE },
      { name: 'Oprettet', property: 'created', filter: Global.searchFilter.DATE_RANGE },
      { name: 'Slettet', property: 'deleted', filter: Global.searchFilter.DATE_RANGE }
    ]
  }
/>
```

Tekst og funktionalitet defineres når komponentet tages i brug.

Eksempelvis “Searchprefs” til dynamisk allokering af menu punkter - hvor hvert objekt i arrayet

```
<PromptButton
    buttonText='Slet reservation'
    modalTitle='Bekræft sletning'
    fullWidth={true}
    buttonColor={'secondary'}
    buttonClass={classes.submit}
    modalButton1Text='Slet'
    modalButton1Function={handleScheduleDelete}
    modalButton2Text='Annuller'
/>
```

Et andet eksempel kunne være vores "bekræftelses" prompt.

Funktionen **"ButtonPrompt"** behandles således de properties der bliver tildelt, og returner efterfølgende en modal

```
<React.Fragment>
    <Button
        fullWidth={props.fullWidth}
        aria-label="edit"
        onClick={handleOpen}
        disabled={props.buttonDisabled}
        variant={props.buttonVariant}
        className={props.buttonClass}
        color={props.buttonColor ?? 'primary'}
        style={{ textTransform: props.textTransform }}
    >
        {props.buttonText}
    </Button>
    <Modal open={open} onClose={handleClose}>
        <div style={{ width: (windowWidth < MAX_WIDTH_FULLSCREEN ? '100vw' : 'auto'), ...modalStyle }}
            className={classes.paper}
        >
            {props.modalTitle && <h2 id="title">{props.modalTitle}</h2>}
            {props.modalText && <p id="simple-modal-description">{props.modalText} </p>}
            <div>
                {props.modalButton1Text &&
                    <Button
                        className={classes.modalButton}
                        variant="contained"
                        color={props.modalButtonColor1 ?? 'primary'}
                        onClick={() => handleClose(1)}>{props.modalButton1Text}
                    </Button>}
                {props.modalButton1Text &&
                    <Button
                        className={classes.modalButton}
                        variant="contained"
                        color={props.modalButtonColor2 ?? 'secondary'}
                        onClick={() => handleClose(2)}>{props.modalButton2Text}
                    </Button>}
            </div>
        </div>
    </Modal>
</React.Fragment>
```

6.7.6 Data Flow og State Management

React benytter sig af et såkaldt “Unidirectional Data Flow[39]”, hvilket på normal dansk betyder data flyder nedad (read-only fra forældre til barn), og events (callbacks) flyder “op” fra barn til forældre.
For Bestilvasketid betyder det bl.a. at vi gemmer vaskeriets data (tider, maskiner, brugere osv.) i applikationens rod, som herfra videresendes til underliggende komponenter (og deres børn). Skulle et barn ønske at opdatere denne data, eksempelvis hente frisk data via vores API, udføres dette via callback funktion modtaget fra roden.

Når data gemmes så højt i hierarkiet introducerer det oftest problemet hvor data skal kanaliseres/håndteres af mange elementer for at nå ud til modtageren (via props). For at undgå dette benytter Bestilvasketid sig af Reacts Context Providers[40], hvor et komponent kan “wrappes” med data, som efterfølgende er tilgængelig for alle komponenters børn og børnebørn. Denne mulighed kaldes også “opt-in[41]”.

```
<Fragment>
  <GlobalContext.Provider value={{
    user: user,
    setUser: setUser,
    laundry: selectedLaundry,
    selectedDate: selectedDate,
    setSelectedDate: setSelectedDate,
    getLaundries: getLaundriesFromAPI,
    history: props.history,
    showSnackbar: showSnackbar,
    closeSnackbar: handleSnackbarClose,
    laundryDateCounter: laundryDateCounter,
    setLaundryDateCounter: setLaundryDateCounter,
    scheduleEnd: scheduleEnd
  }}>

    {showLaundry && < Laundry />}
  </GlobalContext.Provider>
</Fragment>
```

6.7.7 Unit & Integration Test

Kode testing er ikke et nyt begreb, konceptet kan spores tilbage til slut 60erne. Selv manden som i 2003 krediteres for opdagelsen/udviklingen af TDD[42] beskriver det som en “genopdagelse”. Unit og integration tests[43] er dog for mange et overset emne, et unødigt trin som kun introducerer mere arbejde for den stakkels programmør. Sandheden kan dog i mange tilfælde være den modsatte, hvor fejl hurtigere opdages og problemet oftest står mere isoleret. Det er i særdeleshed

[39] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”
[40] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”
[41] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”
[42] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”
[43] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

på sigt at disse tests finder deres succes (gentagende brug), samt i deres kumulative effekt, hvor tusinder af tests kan udføres ved et enkelt tryk.

Bestilvasketid gør brug af Unit/integration test til fejlfinding af komponenter samt projekterede formodninger omkring API'ets virken. Vi benytter Jest[44] som kommer pre-installeret med toolchainen[45] "create-react-app".
Som omtalt handler tests om formodninger. Vi forventer at vores <**PromptButton**> komponent kan gengives på skærmen.

```
test('Promptbutton renders correctly using passed text', () =>
{
  const div = document.createElement('div')
  ReactDOM.render(<PromptButton
    buttonText='Test button'
    modalButton1Text='This is a test button' />, div)
});
```

Vi forventer at modtage vaskeriet data (som array) fra APIet når vi forespørger på en eksisterende pin kode (1234).

```
test('Testing API call, CHECK if laundy is returned on pin "1234 ', async () =>
{
  const response = await APIGetCall(`https://bestilvasketid.ml/api/Laundry/pin/1234`)
  const laundriesArr = Object.values(await response.json())
  expect(laundriesArr).not.toHaveLength(0) // Assumes laundy "1234" exists.
});
```

Ligesom vi forventer en fejlkode hvis vaskeriet/pinkoden ikke eksisterer.

```
test('Testing API call, CHECK if laundry using wrong pin (9999) returns error.', async () =>
{
  const response = await APIGetCall(`https://bestilvasketid.ml/api/Laundry/pin/9999`)
  expect(response.status).toBe(404)
});
```

Test koden kan eksekveres manuelt via kommandoen "**npm test**" - eller automatisk når projektet kompileres, og det er endvidere muligt at inkorporere disse processer i DevOps pipeline.

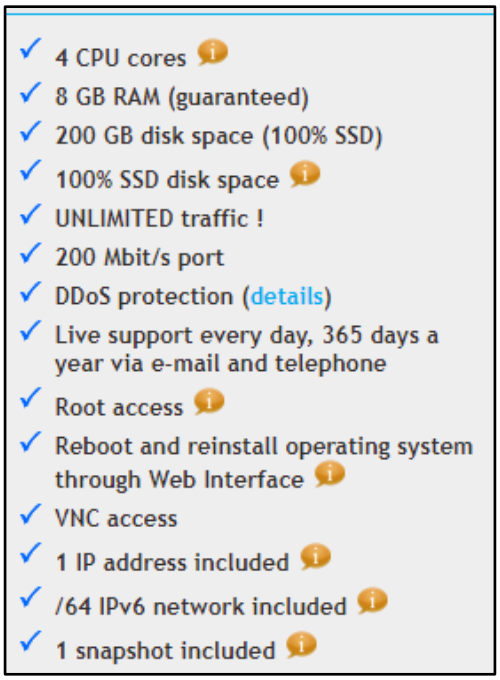
```
PASS  src/App.test.js (8.976s)
  √ Promptbutton renders correctly using passed text (47ms)
  √ AdminCheckbox renders correctly (27ms)
  √ Testing API call, CHECK if pin "1234" exists (2855ms)
  √ Testing API call, GET all users from laundry pin "1234" (228ms)
  √ Testing API call, CHECK if laundy is returned on pin "1234  (161ms)
  √ Testing API call, CHECK if laundry using wrong pin (9999) returns error. (169ms)

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        8.303s
Ran all test suites.

Watch Usage: Press w to show more.
```

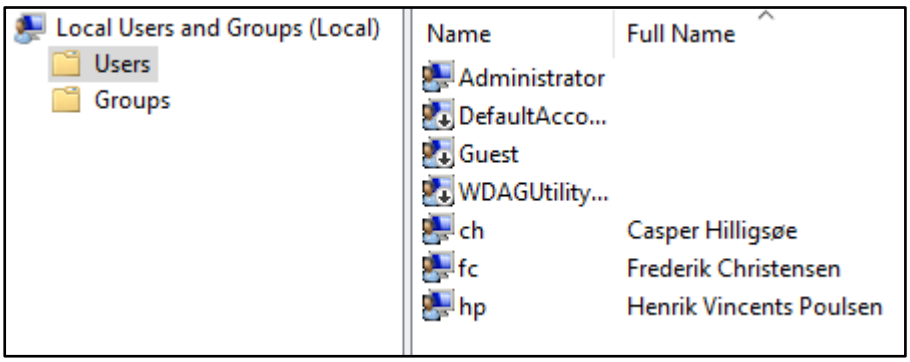
[44] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[45] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

7 Domains & VPS

7.1 Anskaffelse & Windows installation

Vi har valgt at lægge vores system på en "live" server, for derved at kunne teste både i produktionsmiljø (release) og testmiljø (debugging). Efter en del research kom vi frem til Contabo (https://contabo.com), som er en tysk server provider, med kvalitet til discountpriser (5 euro/måned). Fordelen ved at den ligger i Tyskland (mod f.eks. USA eller England) er at der er hurtigere svartider fra Danmark. VPS'en (Virtual Private Server[46]) kom oprindeligt med en linux installation efter eget valg (den fås også med Windows Server til en højere pris), men da vi som elever på TEC har en Windows Server 2019 nøgle, valgte vi at bruge denne. Det var ikke uden problemer, da guides på nettet var lettere mangelfulde, men det lykkedes dog til sidst. Et par administratorer blev installeret, så vi kunne lave Remote Desktop Connection (en indbygget måde i Windows til at fjernstyre andre computere), og derefter påbegynde installering af vigtige komponenter, heriblandt IIS (Internet Information Services).

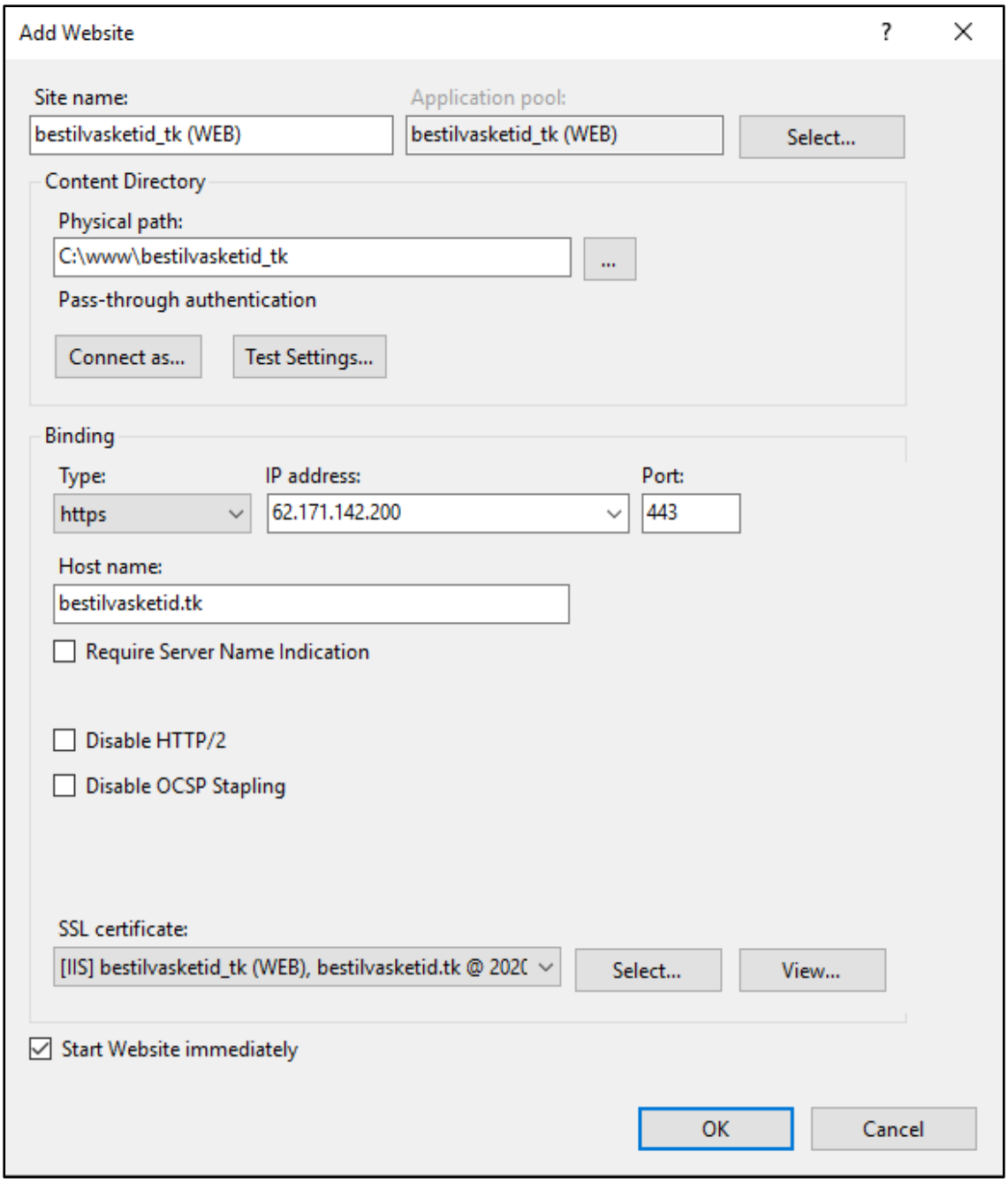




[46] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

7.2 IIS

En af VPS'ens vigtigste funktioner er at fungere som webserver og servere hjemmesiderne (sites) til internettet. Der blev oprettet 2 sites: bestilvasketid_tk (WEB) og bestilvasketid_ml (API). Ideen ved at lave to forskellige sites, er blandt andet at kunne illustrere uafhængigheden af frontend og backend. Dermed åbnes der op for at en kunde ville kunne benytte sig af API'et i egen stående form til f.eks. at lave en special designet hjemmeside. Dette er tiltænkt specielt større boligforeninger.



Hjemmeside domæner blev anskaffet gratis igennem Freenom (https://www.freenom.com) med henholdsvis .tk til frontend og .ml til backend. Disse blev fra Freenom sat til at pege på Contabos Nameservers (ns1.contabo.net, ns2.contabo.net & ns3.contabo.net), og fra Contabos kontrolpanel videresendt til Windows serveren via DNS zoner.

Edit DNS Zone for bestilvasketid.tk

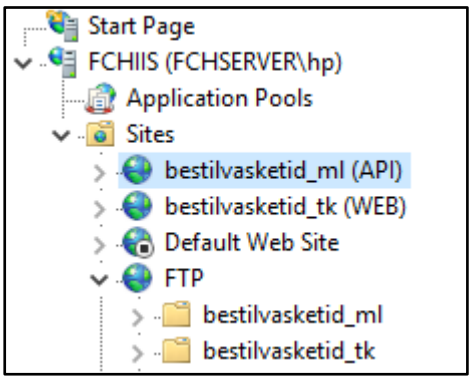
In this section, you can add and remove DNS entries for your domain.

Resource record	TTL	Type	Priority	Data
bestilvasketid.tk	86400	SOA	0	ns1.contabo.net. hostmaster.contabo.de. 2020082601 3600 720...
bestilvasketid.tk	86400	NS	0	ns1.contabo.net
bestilvasketid.tk	86400	NS	0	ns2.contabo.net
bestilvasketid.tk	86400	NS	0	ns3.contabo.net
*.bestilvasketid.tk	86400	A	0	62.171.142.200
bestilvasketid.tk	86400	A	0	62.171.142.200
mail.bestilvasketid.tk	86400	A	0	62.171.142.200
www.bestilvasketid.tk	86400	A	0	62.171.142.200
bestilvasketid.tk	86400	MX	10	mail.bestilvasketid.tk

På Windows serveren bliver domænerne så, via bindings ledt hen til de nyoprettede sites. Både HTTP og HTTPS[47] samt med- og uden www versioner bliver accepteret i browsere, men alle bliver senere redirected til HTTPS uden www.

Type	Host Name	Port	IP Address
http	bestilvasketid.tk	80	*
https	bestilvasketid.tk	443	*
https	www.bestilvasketid.tk	443	*
http	www.bestilvasketid.tk	80	*

Derudover oprettedes også en FTP forbindelse, for i starten let at kunne uploade ændringer til hjemmesiderne via f.eks Visual Studio og File Explorer. Dette blev dog erstattet af Release Pipelinen i Azure DevOps, når der bliver pushet kode til repository.



[47] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

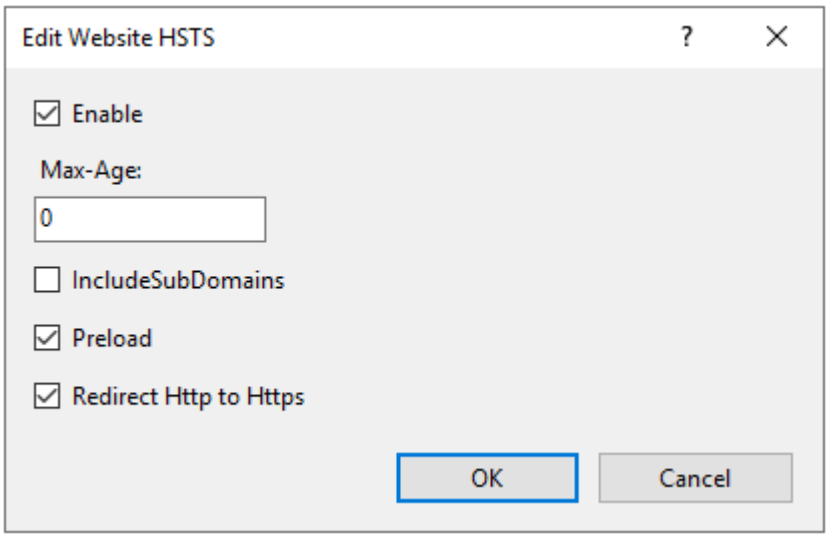
7.3 HTTPS

For at sikre trafikken til og fra serveren, har vi installeret HTTPS (Hypertext Transfer Protocol Secure) over TLS [48](Transport Layer Security). Man kan via Win-ACME få et gratis certifikat samt automatisk opsætning på IIS. Installationsprogrammet køres direkte på Windows serveren fra Windows Powershell.

```
Administrator: Windows PowerShell
PS C:\Users\hp\Downloads\win-acme> .\wacs.exe

 A simple Windows ACMEv2 client (WACS)
 Software version 2.1.10.896 (RELEASE, PLUGGABLE)
 ACME server https://acme-v02.api.letsencrypt.org/
 IIS version 10.0
 Running with administrator credentials
 Scheduled task looks healthy
 Please report issues at https://github.com/win-acme/win-acme

 N: Create certificate (default settings)
 M: Create certificate (full options)
 R: Run renewals (0 currently due)
 A: Manage renewals (5 total)
 O: More options...
 Q: Quit

 Please choose from the menu: N

 Running in mode: Interactive, Simple

  Please select which website(s) should be scanned for host names. You may
  input one or more site identifiers (comma separated) to filter by those
  sites, or alternatively leave the input empty to scan *all* websites.

 5: bestilvasketid_ml (API) (1 binding)
 3: bestilvasketid_tk (WEB) (2 bindings)
 6: spacetravelersguild_ga (2 bindings)
 4: vincentz_tk (1 binding)
 7: vincentz1911_tk (1 binding)

 Site identifier(s) or <Enter> to choose all: 3

 1: bestilvasketid.tk (Site 3)
 2: www.bestilvasketid.tk (Site 3)

  Listed above are the bindings found on the selected site(s). By default all
  of them will be included, but you may either pick specific ones by typing the
  host names or identifiers (comma seperated) or filter them using one of the
  options from the menu.

 P: Pick bindings based on a search pattern
 A: Pick *all* bindings

 Binding identifiers(s) or menu option: 1

 1: bestilvasketid.tk (Site 3)

 Continue with this selection? (y*/n)  - yes

 Target generated using plugin IIS: bestilvasketid.tk

 Requesting certificate [IIS] bestilvasketid_tk (WEB), bestilvasketid.tk
 Store with CertificateStore...
 Installing certificate in the certificate store
 Adding certificate [IIS] bestilvasketid_tk (WEB), bestilvasketid.tk @ 2020-12-10
 Installing with IIS...
 Updating existing https binding bestilvasketid.tk:443 (flags: 1)
 Committing 1 https binding changes to IIS
 Scheduled task looks healthy
 Adding renewal for [IIS] bestilvasketid_tk (WEB), bestilvasketid.tk
 Next renewal scheduled at 2021-2-3 3:21:43
 Certificate [IIS] bestilvasketid_tk (WEB), bestilvasketid.tk created
```

[48] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

I .NET Core er der brugt **app.UseHttpsRedirection();** i Startup.cs -> Configure()
og for at få React front-enden til at redirecte til HTTPS har vi brugt HSTS[49] (HTTP Strict Transport Security) på IISen.



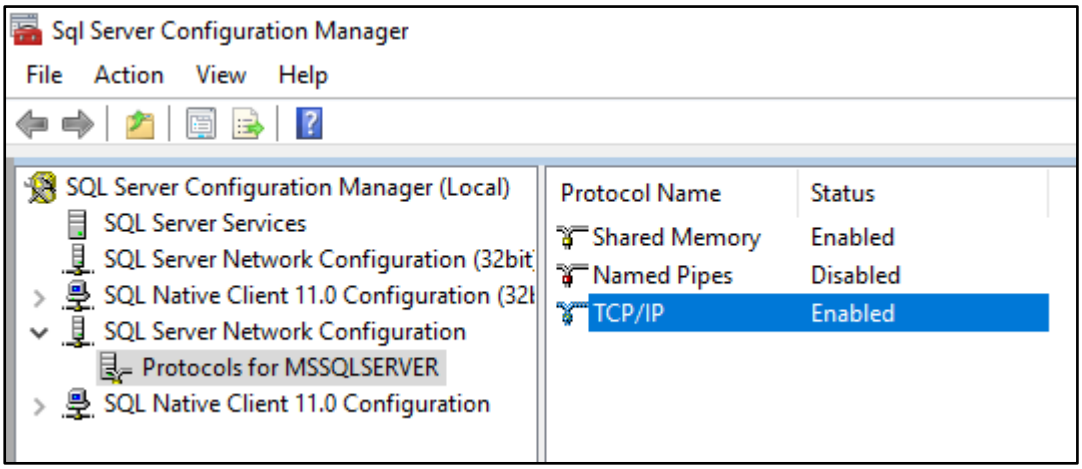
[49] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

8 SQL

Ofte kan det være en god ide at starte et projekt ud med at lave et udkast af modeller i databasen som tabeller, da det hurtigt kan give et godt overblik over hvilke elementer der skal arbejdes med. I vores tilfælde startede vi ud med en meget kompliceret database som brainstorm, og skar derefter fra, hvad der var unødvendigt.

8.1 Installering

Selve database setuppet består af en Microsoft SQL Server Express (64-bit) (MSSQL) version 15.0.2070.41, som i Sql Server Configuration Manager har fået åbnet adgang via TCP/IP på IP 62.171.142.200 port 1433, for at kunne tilgås fra eksterne netværk.



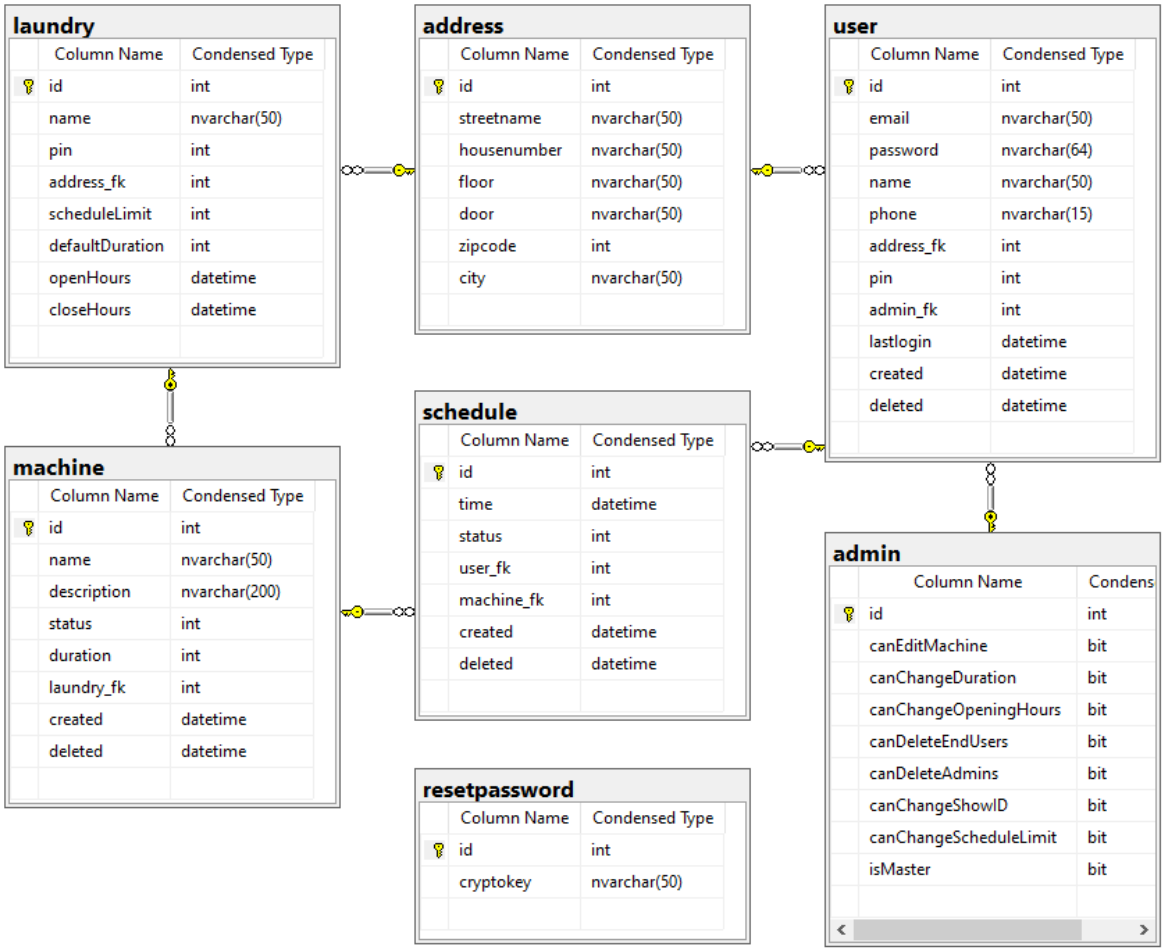
Derefter blev der åbnet op for port 1433 på serverens Windows Firewall.

Med oprettelse af en database kaldet dbbestilvasketid.dk, kunne vi udlede at SQL Connection String'en ville være følgende:

```
Data Source=62.171.142.200;Initial Catalog=dbbestilvasketid.dk;
User ID=sa;Password=***********
```

8.2 Tabeller og kolonner

Som man kan se, så ligner tabellerne i databasen meget de modeller vi har i API'et. Dette er helt bevidst, da vi vil kunne forenkle dataoverførsler via et C# Datatable og konvertere dem til lister af C# objekter. I nedenstående database diagram er de forskellige tabeller vist med kolonner og tilhørende nøgler, relationer og typer.



8.3 Primary Keys

Primary Keys er alle af INT typen, og alle navngivet [id]. For at kunne skelne mellem de forskellige id'er, har vi inddelt Identity Seed'et (det id som automatisk tildeles og forøges med 1 ved oprettelse af en nye tilføjelser i tabellerne) i forskellige segmenter:

Admin: 100
Laundry: 1.000
Machine: 10.000
User: 100.000
Schedule: 1.000.000

8.4 Foreign Keys

_fk er Foreign Keys som relaterer til id'er i andre tabeller. Eksempelvis har et [laundry] et [address_fk] som relaterer til et id i [address] tabellen, mens en [machine] har et [laundry] id.
Vi benytter os af Foreign Key Constraints for at forhindre ødelagte forbindelser mellem tabeller og forhindre at ugyldige data indsættes i Foreign Key kolonnen.

8.5 Typer

SQL typerne (Condensed Type i diagrammet), ligner meget C# typer, med *nvarchar* som værende en string type og *bit* værende en boolean type. Specielt med henblik på datetime bliver det derved meget nemmere at konvertere data mellem C# og SQL.

9 Backend

Vi har valgt at bruge .NET Core 3.1, af følgende årsager:

- Det er gennemtestet.
- Der er masser af ressourcer på nettet.
- Vi har brugt det i tidligere sammenhæng.
- Det benytter sig af C# som vi alle er familiære med.

9.1 Models

Vi har oprettet modeller, som er simple klasser med properties (egenskaber), så disse ligner tabellerne i SQL.

Eksempel på en model med DTO[50]:

```
namespace bestilvasketid_APIv2.Models
{
    public class Laundry
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Pin { get; set; }
        public int Address { get; set; }
        public int ShowID { get; set; }
        public int ScheduleLimit { get; set; }
        public int DefaultDuration { get; set; }
        public DateTime OpenHours { get; set; }
        public DateTime CloseHours { get; set; }
    }

    public class Laundry_DTO
    {
        public Laundry Laundry { get; set; }
        public Address Address { get; set; }
        public List<Machine_DTO> Machines { get; set; }
    }
}
```

[50] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

9.2 API (Application Programmable Interface)

Når valget kom til data transport mellem server og frontend, stod vi overfor et relativt let valg. Med et RESTful API[51] var det nemt at implementere en løsning både på frontend i React og i backend med .NET Core. Syntax i JSON er relativt let forståeligt, da det modsat gRPC[52] er i tekstformat, og det er en udbredt løsning, så der er masser af support at hente. Microsoft har en skabelon med et simpelt API man kan starte ud fra. Vi har bygget videre på skabelonen ved at tilføje mere sikkerhed og asynkronicitet[53] m.m. API'et blev opbygget med de nødvendige GET-, POST-, PUT- og DELETE kald der var nødvendige. Når der er flere ting der skal sendes med samtidigt, bliver der oprettet DTO'er (Data Transfer Objects). DTO'erne består ofte af flere C# modeller og lister deraf.

Eksempel på API kald for Users:

METHOD	URL	DESCRIPTION	BODY	RESULT
POST	*/api/User	Creates a new User	User DTO	Ok(id of new User)
GET	*api/User/{PIN}	Reads Users by PIN in URL		Ok(List of User DTOs)
POST	*/api/User/login	Reads Non-deleted User by Email in body	User (email & password)	Ok(User DTO)
POST	*/api/User/reset	Sends Reset email to User by Email in body	User (email)	Ok
POST	*/api/User/reset/{KEY}	Resets User Password with Key in URL	User (password)	Ok
PUT	*/api/User/reset	Updates User Password	User (id, password)	Ok
PUT	*/api/User	Updates a User	User DTO	Ok(User DTO)
DELETE	*/api/User/{USER ID}	Deletes a User by ID in URL		Ok
POST	*/api/User/Undelete/{USER ID}	Undeletes a User by ID in URL		Ok

API kaldet fra frontenden bliver modtaget som HTTP requests i controllerne og via implementerede dependency injected interfaces, ledt til services, hvor dataoverførsler fra databasen eksekveres via SQL kommandoer.

Vi har oprettet en hjemmeside på .NET Core serveren til API dokumentation på https://bestilvasketid.ml

[51] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[52] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[53] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

User Admin Address Laundry Machine Schedule

User API (id: xxx.xxx)

METHOD	URL	DESCRIPTION	BODY	RESULT
POST	bestilvasketid.ml/api/User	Creates a new User	User DTO	Ok(id of new User)
GET	bestilvasketid.ml/api/User/{PIN}	Reads Users by PIN in URL		Ok(List of User DTOs)
POST	bestilvasketid.ml/api/User/login	Reads Non-deleted User by Email in body	User (email & password)	Ok(User DTO)
POST	bestilvasketid.ml/api/User/reset	Sends Reset email to User by Email in body	User (email)	Ok
POST	bestilvasketid.ml/api/User/reset/{RESET KEY}	Resets User Password by Password in body & Key in URL	User (password)	Ok
PUT	bestilvasketid.ml/api/User/reset	Updates User Password	User (id, password)	Ok
PUT	bestilvasketid.ml/api/User	Updates a User (email, phone, name, pin, admin from User, and address from User DTO)	User DTO	Ok(User DTO)
DELETE	bestilvasketid.ml/api/User/{USER ID}	Deletes a User by ID in URL		Ok
POST	bestilvasketid.ml/api/User/Undelete/{USER ID}	Undeletes a User by ID in URL		Ok

User


```
{
  "id": 106065,
  "name": "bob bobson",
  "phone": "123132123",
  "email": "bob@bob21.dk",
  "password": "Passw0rd",
  "lastLogin": null,
  "created": "2020-09-24T23:22:38.88",
  "deleted": null,
  "pin": 1234,
  "admin": null,
  "address": 66
}
```


User DTO


```
{
  "id": 100000,
  "user": User,
  "admin": Admin,
  "address": Address
}
```


9.3 SQL Database access

Valget mellem at benytte et framework som Entity Framework, at bruge “Linq to SQL” eller at benytte SqlClient var ikke let. Der er fordele ved alle, men i sidste ende faldt valget på sidstnævnte, da de andre metoder alligevel bruger SqlClient, blot pakket ind, og det giver os derfor mere kontrol over koden.
En metode til at konvertere et Datatable til en liste af objekter blev finpudset og optimeret, og en service for hver af tabellerne i databasen blev oprettet med CRUD metoder (Create/Read/Update/Delete).
Samtidigt benytter vi os af parametre i sql kommandoerne, for at undgå SQL-injection[54], som potentielt ville kunne skade databasen.

Hver service implementerer et interface, som bliver injected i controlleren med dependency injection. Med DI (dependency injection) kan man skabe afhængigheder (dependencies) uden for den klasse, der bruger den. Man injecter (indsprøjter) services udefra og tager dermed kontrol over deres oprettelse væk fra selve klassen. I .NET Core kan der benyttes tre mulige løsninger: Singleton, Transient eller Scoped i ConfigureServices metoden i Startup.cs.

Singleton opretter en enkelt instans gennem programmets levetid.
Transient er altid forskellige; en ny instans oprettes til hver controller og service.
Scoped opretter en instans for hvert scope. En request er et scope, og dermed genbruges instansen i begrænset omfang.

```
services.AddSingleton<ISqlService, SqlService>();
services.AddScoped<IUserService, UserService>();
...
```

[54] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

Ved SQL servicen benytter vi os af singleton som anbefales da der er begrænset IO adgange, og i de resterende services bruger Scoped for bedre garbage collection (GC) og memory management.

9.4 Hashing/Salting Passwords

For at optimere sikkerheden mest muligt, valgte vi at hashe[55] og salte[56] kodeord i databasen med PBKDF2[57] (Password-Based Key Derivation Function 2). PBKDF2 er en anerkendt krypteringsmetode og benyttes af bla. Apples IOS, Microsoft, Winzip og Wifi (WPA and WPA2).

Ved hjælp af RNGCryptoServiceProvider, som er en del af System.Security.Cryptography, laver vi et salt, som er en række tilfældige data (RNG står for Random Number Generator), og bruger derefter dette sammen med kodeordet i metoden "Rfc2898DeriveBytes", for at få lavet en nøgle. Fordelen med at bruge et tilfældigt salt sammen med krypteringen, er hvis der findes to ens kodeord i databasen og de kun er krypteret, ville de være identiske. Med et tilfældigt salt, vil begge kodeord være forskellige.
Ulempen ved hashing er at det er envejs kryptering. Man kan kun sammenligne de to nøgler som er resultatet af krypteringen. Derfor vil kodeordet ikke kunne genskabes i tekstform. Hvis en bruger har glemt sit kodeord, er den eneste løsning at oprette et nyt.

Eksempel på passwords i databasen:

	id	email	password
1	100001	hpoulsen76@gmail.com	$HASH|V1$10000$Mlg4XVCcdrtvkB9ATxbjTW/5cfwKTk7PhzjYeg==
2	100002	c@c.dk	$HASH|V1$10000$PINQeMjlGGLFA/1ALEdMYhOkbjJHfa+2faUpvg==
3	100010	fchristensen26@gmail.com	$HASH|V1$10000$EwmM8o9x6GA4eQdhwarBsqg03q6JARrnwDBGUA==
4	100020	trash@scumworks.com	$HASH|V1$10000$IWGrt8htOxZkPQdkwMmha63LQeqJn1CM4yLszQ==
5	100028	bestilvasketid@gmail.com	$HASH|V1$10000$SjJ3+UZa1mlOGUtJ0ykweccfjoqKN0ox8BDcgw==

9.5 Scheduled Tasks

I forbindelse med oprydning i databasen, har vi oprettet en task som kører dagligt og sletter vasketider, som er mere end 3 måneder gamle, samt brugere som har været inaktive i 365 dage, i forhold til GDPR[58] love og regler.

Timeren startes i Startup.cs:

```
Timer cleanupTimer = new Timer(async (e) =>
{
```

[55] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[56] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[57] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[58] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

```
    List<User> deletedUsers = await new SqlService().CleanupDatabaseAsync(365, 90);
    new MailService().SendDeletedUserEmail(deletedUsers);
}, null, 10 * 1000, 24 * 60 * 60 * 1000); // Start delay = 10 seconds, Runs every 24 hours
```

Og bliver eksekveret i SqlService.cs:

```
public async Task<List<User>> CleanupDatabaseAsync(int userDays, int scheduleDays)
{
  SqlCommand cmd = new SqlCommand("ALTER TABLE [user] nocheck constraint all;" +
     "ALTER TABLE [schedule] nocheck constraint all;" +
     $"DELETE FROM [user] OUTPUT DELETED.* WHERE lastlogin <= DATEADD(day, -{userDays}, GETDATE()) ;" +
     $"DELETE FROM [user] OUTPUT DELETED.* WHERE lastlogin IS NULL AND created <= DATEADD(day, -
{userDays}, GETDATE());" +
     $"DELETE FROM [schedule] WHERE time <= DATEADD(day, -{scheduleDays}, GETDATE())" +
     "ALTER TABLE [user] check constraint all;" +
     "ALTER TABLE [schedule] check constraint all;");
     eturn await Command2List<User>(cmd);
}
```

9.6 CORS (Cross-origin resource sharing)

For at kunne hente API data på tværs af domæner, skal man benytte CORS (Cross-Origin Resource Sharing)[59]. Dette er blevet sat op i startup.cs filen i .NET Core.
Der er mulighed for at begrænse både hvorfra (origin), hvilken type (method) og hvilken header der bliver sendt.

Først med en policy i ConfigureServices():

```
services.AddCors(options =>
        {
           options.AddPolicy(name: "CorsPolicy",
              builder =>
              {
                 //builder.WithOrigins("https://bestilvasketid.tk", "https://bestilvasketid.ml");
                 builder.AllowAnyHeader();
                 builder.AllowAnyOrigin();
                 builder.AllowAnyMethod();
              });
        });
```

Derefter startes den i Configure():

```
app.UseCors()
```

I API'et skal policy'en tilføjes som en attribute på controller-klassen:

```
[EnableCors("CorsPolicy")]
```

[59] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

I ovenstående eksempel tillader vi alt, men har forberedt en begrænsning til kun at godkende API kald der kommer fra følgende domæner: "https://bestilvasketid.tk", "https://bestilvasketid.ml".

9.7 Asynkronicitet

I gamle dage startede et program fra toppen og sluttede i bunden. Senere kom OOP[60] (object oriented programming) til og gjorde det muligt at oprette instanser af klasser, samt genbruge disse og deres metoder ved hjælp af nedarvning, polymorfisme, abstraktion osv.

Så sent som i 2012 blev asynkrone kald[61] implementeret i C# da version 5.0 blev introduceret. Man kunne nu give programmet mulighed for at udføre anden kode, mens man venter på at en længerevarende asynkron opgave gennemføres. Da dataoverførsler over internettet ofte resulterer i en forsinket respons (ping og overhead[62]), er det oplagt at lave API kald asynkrone, så serveren ikke står og hænger, mens den venter på svar fra. f.eks databasen.

Nedenstående billede viser at vi henter alle brugerne fra databasen via en asynkron opgave med [async Task], og afventer svar med [await]:

```
[HttpGet]
public async Task<List<User>> GetAll() { return await _userService.ReadAllUsers(); }
```

Alle services der henter data fra databasen er også gjort asynkrone.

9.8 Authentication / API Key

Uden en form for authentication ville alle der har API'ets endpoint (URLs), frit kunne hente og ændre data. Ofte ser man at specielt open source API'er, tilgås ved at tildeles en API key ved signup, og at denne bliver sat ind i URL'en sammen med andre parametre. Dette frarådes stærkt, da det er sikkerhedsmæssigt uforsvarligt, da URL'er er mere blottede på f.eks. client maskinen.

Vi har i stedet valgt at inkludere en API nøgle i headeren på alle API kald. En effektiv måde at tjekke om headeren indeholder API-nøglen er ved at oprette en **[attribute]**. Denne kan enten tilføjes for alle API kald i controlleren, eller ved enkelte kald, hvis man ønsker at nogle kald skal have speciel godkendelse/autorisering.

Attributen tjekker om der er et korrekt nøglepar. Først om der er en key i headeren der hedder "ApiKey" og derefter om valuen for ApiKey er identisk, med den værdi der er sat i appsettings.json.

[60] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[61] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"
[62] Se "Definitioner og ordforklaringer" i "Procesrapport bilag"

```
public class ApiKeyAuthAttribute : Attribute //, IAsyncActionFilter
  {
    public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
    {
      //If the header doesn't have value "ApiKey" then return Unauthorized(401)
      if (!context.HttpContext.Request.Headers.TryGetValue("ApiKey", out var potentialApiKey))
      {
        context.Result = new UnauthorizedResult();
        return;
      }

      //If the header key value doesn't equals secret apikey then return Unauthorized(401)
      string apiKey = context.HttpContext.RequestServices.GetRequiredService<IConfiguration>().GetValue<string>("ApiKey");
      if (!apiKey.Equals(potentialApiKey))
      {
        context.Result = new UnauthorizedResult();
        return;
      }

      await next();
    }

  }
```

9.9 IP RATE LIMITING

For at forhindre DDOS-[63] og Brute Force angreb[64] mod databasen, har vi installeret RateLimit, som er en middleware i .NET Core.

Den er pt. sat til 60 API kald pr IP i minuttet og kan justeres i appsettings.json.

```
"IpRateLimit": {
  "EnableEndpointRateLimiting": true,
  "StackBlockedRequests": false,
  "RealIPHeader": "X-Real-IP",
  "ClientIdHeader": "X-ClientId",
  "HttpStatusCode": 429,
  "GeneralRules": [
    {
      "Endpoint": "*:/api/*",
      "Period": "1m",
      "Limit": 60
    }
  ]
}
```

[63] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

[64] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”

9.10 MAILSERVICES

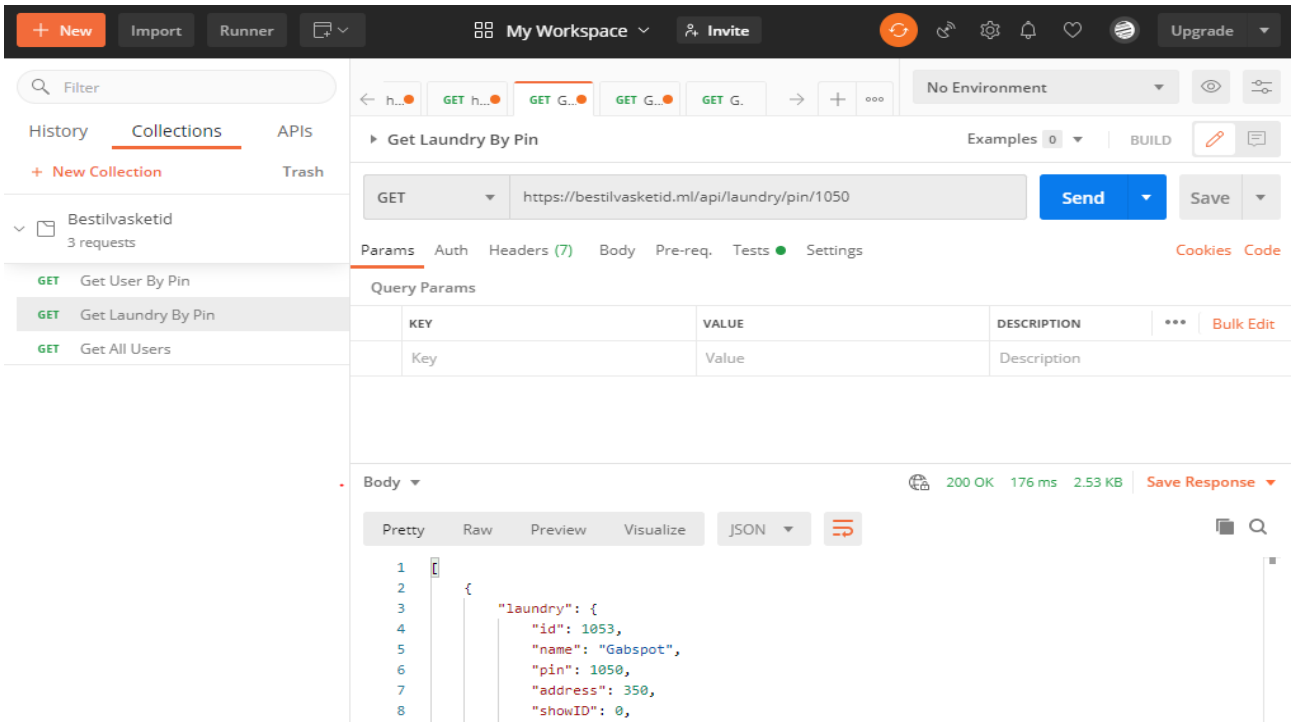
For at huske brugere på deres bestilte vasketider og advisere administratorer om bruger rapporterede fejl på maskinerne, har vi oprettet en mailservice, som via en Gmail konto (bestilvasketid@gmail.com) sender mails ud.

- **Vasketids påmindelse** - 30 minutter før påbegyndte vasketider.
- **Annulleret vasketid** - I tilfælde af administrator har annulleret vasketid.
- **Fejlmeldt maskine** - Bruger-rapporteret fejl på maskine sendt til administratorer.
- **Reset Password** - Sender et link til bruger som kan resette brugerens kodeord.
- **Slettet Bruger** - Sender en besked til bruger, om at deres konto er blevet slettet.

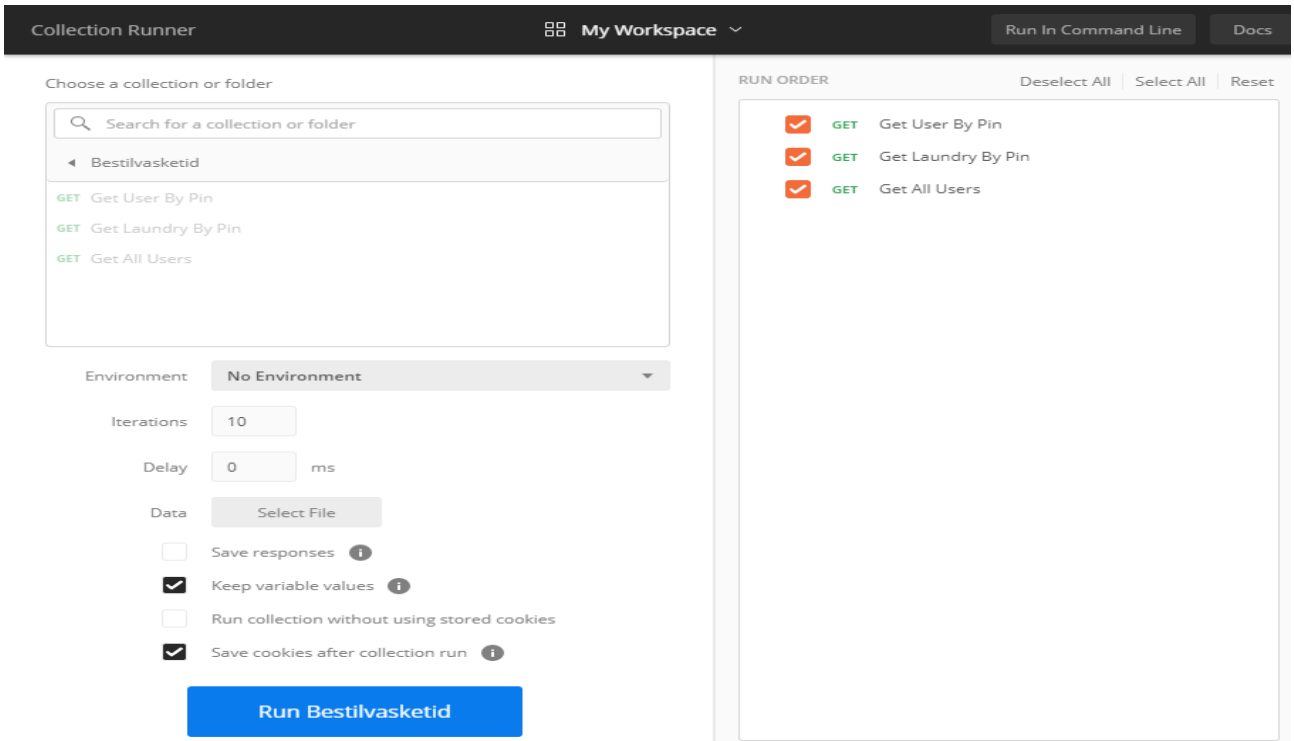
9.11 Tests

Postman (https://www.postman.com) har været vores mest brugte testværktøj. Med ganske få klik kan man hurtigt teste API'et og dermed også database-forbindelsen. Dette kan gøres både i debugging på localhost, men også op mod release build'et på bestilvasketid.ml.

Vi har oprettet "Collections" i Postman, som er en samling af API kald.



Disse kan så bruges i en "Collection Runner" til at køre alle testene over flere gange (iterations).



Resultaterne med response kode, responstid og antal KB får man så her:

Collection Runner | Run Results | My Workspace | Run In Command Line | Docs

40 PASSED | 0 FAILED | Bestilvasketid | No Environment | just now | Run Summary | Export Results | Retry | New

Iteration 9

GET	Get User By Pin	https://bestilvasketid.ml/...	/ Get User By Pin	200 OK	231 ms	615.102 KB
	Status code is 200				
	Status code is 200				
GET	Get Laundry By Pin	https://bestilvasketid.ml/...	/ Get Laundry By Pin	200 OK	67 ms	2.255 KB
	Status code is 200				
GET	Get All Users	https://bestilvasketid.ml/...	/ Get All Users	200 OK	375 ms	615.102 KB
	Status code is 200				

Iteration 10

GET	Get User By Pin	https://bestilvasketid.ml/...	/ Get User By Pin	200 OK	341 ms	615.102 KB
	Status code is 200				
	Status code is 200				
GET	Get Laundry By Pin	https://bestilvasketid.ml/...	/ Get Laundry By Pin	200 OK	76 ms	2.261 KB
	Status code is 200				
GET	Get All Users	https://bestilvasketid.ml/...	/ Get All Users	200 OK	247 ms	615.102 KB
	Status code is 200				

1 2 3 4 5 6 7 8 9 10

10 FREMTID/AFGRÆNSNING

Vi har under projektets forløb valgt at afgrænse os, i henhold til visse punkter beskrevet i kravspecifikationen.

Muligheden for at tjekke maskinernes status via tilsluttet/påmonteret Arduino, er flyttet til et fremtidigt projekt. Frontend/backend vil dog nemt kunne implementere den tiltænkte løsning, uden de store ændringer eller rettelser.

Vi har primært valgt at fokusere på de tekniske aspekter som indgår i uddannelsen. Business case, markedsundersøgelser samt meget specifikke juridiske aspekter af GDPRs lovgivning er udeladt, eftersom der oftests kræves specialiseret viden, som ligger uden for vores kompetencer. Vi har gjort tiltag for øget sikkerhed og håndtering af personfølsomme data, men ikke altid i fuldkommen overensstemmelse med de gældende standarder og reglementer.
Vi vil tilkalde professionel assistance, således at vi kan sikre disse ting er gennemarbejdet, inden en evt. lancering af vaskesystemet gennemføres.

10.1 FUNKTIONELLE KRAV

Vi har valgt at tillade brugere genskaber deres password med deres originale password. Kryptering af e-mails er ligeledes rykket frem til hjemmesiden bliver lanceret, da logins har været sværere at søge/sammenligne under test.
Google Analytics har vi valgt ikke skal indgå i projektet, da det først har relevans når/hvis hjemmesiden bliver lanceret til offentligheden.
Vi har fravalgt en brugervejledning/FAQ på hjemmesiden, da vi mener hjemmesidens design og funktioner, er tilpas brugervenlig og intuitiv uden.

11 Konklusion

På baggrund af vores gruppearbejde, projektets udformning samt opstillede problemformulering kan vi konkludere, at vores målsætning om at undersøge-

1. Hvordan man laver et konkurrencedygtigt cross-platform system, som er nemt og hurtigt at bruge for både administratorer og brugere.
2. Hvordan man udvikler man et sådant system, så det er sikkert med mange simultane brugere og med høj driftssikkerhed

begge er opfyldt. Begrundelsen for denne konklusion er uddybet nedenfor.

11.1 Gruppearbejde

Vi har haft en god synergi og forståelse i gruppearbejdet, som vi mener kommer af dialog ved daglige statusmøder, samt fremvisning af entusiasme for svære spørgsmål og idéer.
Dette resulterede i høj motivation, på trods af udfordringer, som vi i første omgang ikke kunne løse. Som gruppe har vi arbejdet på projektet - både kollektivt og individuelt - med daglig kontakt vedrørende spørgsmål og statusmøder.
Vi mener, at effektiviteten af dette har højnet det individuelle arbejde. Arbejdsplanlægningen, samt status heraf, har været positivt.

11.2 Projektstruktur

Igennem disse fire uger har vi haft stort fokus på projektstrukturen, og her har DevOps været et effektivt værktøj. DevOps har gjort det muligt at facilitere agile arbejdsmetoder, der har resulteret i mere produktivt arbejde, højere kvalitet og med et overblik der konstant har været tilstedeværende.

11.3 Idéen

Med udgangspunkt i projektidéens oprindelse, nemlig at mange vaskeriers nuværende løsninger er enten tavle eller logbog ved reservation af vasketider, udarbejdede vi en hjemmeside med tilhørende app, som er designet til at være let tilgængelig, med stor fokus på bruger- og prisvenlighed. Derfor mener vi, at vores formål med produktet er lykkedes.

11.4 Opsummering

Med udgangspunkt i ønsket om at udvikle et produkt, som er nemt og hurtigt for hhv.

administratorer og brugere, har vi udarbejdet en hjemmeside og app, der har særligt fokus på farver samt størrelse af elementer. Dette har til formål at gøre det let læseligt og intuitivt, særligt hvor og hvad man klikker på. I vores design har vi haft fokus på UX[65], hvilket har gjort at brugere og administratorer har en bedre brugeroplevelse. Dette er gjort ved at fjerne så mange klik for at udføre en handling som muligt. Tilmed er det blevet lettere for administratorerne, at håndtere diverse udfordringer og spørgsmål brugeren kommer med.

Et vigtigt element i at skabe et konkurrencedygtigt produkt er lave omkostninger. Dette er opnået ved ikke at tilføje krav om, at kunden skal købe digitalt integrerede tørre- og vaskemaskiner, der ofte ses hos andre vaskesystemer. Serveren vi har købt fra Contabo er billig i drift og der skal mange tusinde vaskerier til, før vi behøver at tænke på at udvide den med flere ressourcer. Samtidig er produktet en opgradering fra de snart forældede vaske- og logtavler. En fordel er samtidig, at der nærmest intet opstartsarbejde er forbundet med hertil. Boligforeninger der bestiller vores system, skal udelukkende sende en pinkode ud til deres beboere, som så selv opretter sig på hjemmesiden eller appen. Vi vurderer dermed, at det er lykkedes at skabe et konkurrencedygtigt system, som nemt og hurtigt kan benyttes af både administratorer og brugere fra forskellige enheder.

Vi har arbejdet på at udvikle et vaskesystem, der kan håndtere adskillige brugere samtidig, samt har et højt sikkerhedsniveau . Dette ønske mener vi er opfyldt, da vi har sat hjemmesiden op med HTTPS, samt har sikret passwords med kryptering. Ydermere har vi valgt en serverudbyder, der garanterer 99,99% oppetid. Vi har lavet asynkrone metoder i koden, som gør at der kan foretages flere handlinger i koden på samme tid, hvilket resulterer i hurtigere hastighed på hjemmesiden.

[65] Se "Definitioner og ordforklaringer" i “Procesrapport bilag”